DBT_GUIDE

Helios â€” dbt Engineering Guide

`DBT_GUIDE.md` Â· Companion to `HELIOS_PROJECT_BIBLE.md` Â§15â€“Â§17 (Analytics Engineering, dbt, BigQuery) and Â§8 (Data Model) Â· **Version:** v1.0 Â· **Date:** 2026-06-03

Purpose. This is the production-grade analytics-engineering handbook for Helios â€” the standalone reference for the dbt layer that transforms raw GA4 events into governed marts. Those marts are the foundation of the entire product: the semantic layer (`models/semantic/semantic_layer.yaml`) exposes them as metrics, and the agents compose those metrics via `semantic-mcp` â€” they never hand-write SQL. Correct, tested, documented, fresh marts are therefore non-negotiable. It is written assuming Helios will eventually run **production-grade and multi-tenant**, while staying honest about the static public sample dataset it is currently built on.

How to use this guide

- Build in dependency order (`DEPENDENCY_MAP.md` M1â€“M5): sources â†' staging â†' intermediate (the two keystones) â†' marts â†' semantic layer.
- This guide is the *how* of the dbt layer; `CLAUDE.md` is the operating rules, the Bible is the *why*, `METRIC_GOVERNANCE_GUIDE.md` owns the metric definitions downstream of these marts.
- Every code block is real and copy-usable. The keystones (sessionization, `reached_*` monotonicity, revenue reconciliation) fail **silently** â€” write their golden tests first (Â§6).

Conventions cheat-sheet

Area	Rule
Layers / prefixes	`stg_<source>__<entity>` â†' `int_<source>__<entity>` â†' `fct_*` / `dim_*`; one model per file; no cross- or upward-layer refs; snake_case
Materializations	staging = `view`, intermediate = `ephemeral`, marts/core = `incremental` (`insert_overwrite`), finance/growth/dims = `table`, semantic = `view`
Partition / cluster	core facts: `partition_by event_date` (DATE, day) + `cluster_by [device_category, channel_group]`; `require_partition_filter` on large facts
Incremental	`insert_overwrite` + 3-day `is_incremental()` lookback (re-materializes recent partitions for late shards)
Session key	`session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))`; `sessions = COUNT(DISTINCT session_key)`
Funnel flags	`reached_*` are **max-downstream monotonic** â†' `sessions â‰¥ reached_view_item â‰¥ â€¦ â‰¥ reached_purchase`; step rates â‰¤ 1 by construction (`did_*` retired)
Engaged session	`session_engaged = '1' OR engagement_time_msec ≥ 10000`
Channel grouping	exactly 10 groups, defined in **one** macro `channel_group_case()`; `traffic_source` is user first-touch, so prefer session-scoped `event_params` source/medium
Money / rates	`*_in_usd` columns only; rates as `SUM(num)/SUM(den)` after grouping (Simpson's-paradox defense)
Marts shape	**wide** (denormalized with descriptive dims) so the semantic layer slices without runtime joins

Table of Contents

6. Testing Strategy
7. Freshness Strategy
8. Lineage Strategy
9. Documentation Strategy
10. Production-Readiness Checklist

1. dbt Project Structure

Helios is an ELT system whose entire "T" is one dbt project. The repository layout is not cosmetic: it encodes the layered DAG (`src_ga4 → stg_ga4__* → int_ga4__* → fct_*/dim_* → semantic_layer.yaml`), the materialization strategy, and the governance boundary that keeps the agents on rails. Every file below maps to a pinned artifact in the model catalog; nothing is invented at run time.

Project tree

```
helios/
â”œâ”€â”€ dbt_project.yml                # project config: paths, folder-level +configs, vars
â”œâ”€â”€ packages.yml                   # dbt-utils, dbt_expectations, dbt_project_evaluator, codegen
â”œâ”€â”€ profiles.yml                   # dev→helios_dev (oauth), prod→helios_prod (WIF/SA)
â”œâ”€â”€ selectors.yml                  # named selectors (slim CI, layer builds)
â”œâ”€â”€ seeds/
â”‚   â””â”€â”€ channel_group_mapping.csv    # canonical source→channel mapping (feeds channel_group_case)
â”œâ”€â”€ macros/
â”‚   â”œâ”€â”€ get_event_param.sql          # get_event_param(key, type) typed extraction
â”‚   â”œâ”€â”€ sessionize.sql               # sessionize(): session_key = TO_HEX(MD5( ... ))
â”‚   â”œâ”€â”€ channel_group.sql            # channel_group_case(): SINGLE SOURCE OF TRUTH, 10 groups
â”‚   â””â”€â”€ test_revenue_reconciles.sql  # custom generic test: mart total == source total
â”œâ”€â”€ models/
â”‚   â”œâ”€â”€ staging/
â”‚   â”‚   â”œâ”€â”€ _ga4__sources.yml        # src_ga4 declaration (sources + freshness)
â”‚   â”‚   â”œâ”€â”€ _ga4__models.yml         # staging schema.yml: docs, tests, columns
â”‚   â”‚   â”œâ”€â”€ _ga4__docs.md            # doc blocks reused across layers
â”‚   â”‚   â”œâ”€â”€ stg_ga4__events.sql      # view: 1:1 typed/renamed events
â”‚   â”‚   â””â”€â”€ stg_ga4__event_params.sql# view: unnested event_params long table
â”‚   â”œâ”€â”€ intermediate/
â”‚   â”‚   â”œâ”€â”€ _int_ga4__models.yml     # intermediate schema.yml (tests, NOT exposed to BI)
â”‚   â”‚   â”œâ”€â”€ int_ga4__sessionized.sql # KEYSTONE: session grain, channel/landing/source
â”‚   â”‚   â””â”€â”€ int_ga4__funnel_steps.sql# KEYSTONE: reached_* monotonic flags + session_revenue
â”‚   â”œâ”€â”€ marts/
â”‚   â”‚   â”œâ”€â”€ core/
â”‚   â”‚   â”‚   â”œâ”€â”€ _core__models.yml     # contracts, tests, exposures-facing docs
â”‚   â”‚   â”‚   â”œâ”€â”€ fct_sessions.sql      # incremental: session PK session_key
â”‚   â”‚   â”‚   â”œâ”€â”€ fct_funnel.sql        # incremental: primary session grain for semantic
â”‚   â”‚   â”‚   â”œâ”€â”€ fct_daily_funnel.sql  # incremental: additive day x dim step counts
â”‚   â”‚   â”‚   â”œâ”€â”€ dim_users.sql         # table: user_pseudo_id (PK user_key)
â”‚   â”‚   â”‚   â”œâ”€â”€ dim_items.sql         # table: item_id (PK item_key)
â”‚   â”‚   â”‚   â”œâ”€â”€ dim_channels.sql      # table: 10 channel groups (PK channel_key)
â”‚   â”‚   â”‚   â””â”€â”€ dim_date.sql          # table: conformed date spine
â”‚   â”‚   â”œâ”€â”€ finance/
â”‚   â”‚   â”‚   â”œâ”€â”€ _finance__models.yml
â”‚   â”‚   â”‚   â”œâ”€â”€ fct_orders.sql        # table: transaction_id (PK order_key)
â”‚   â”‚   â”‚   â””â”€â”€ fct_order_items.sql   # table: order line (PK order_item_key)
â”‚   â”‚   â””â”€â”€ growth/
â”‚   â”‚       â”œâ”€â”€ _growth__models.yml
â”‚   â”‚       â”œâ”€â”€ fct_funnel_by_dim.sql # table: funnel rollup by canonical dimension
â”‚   â”‚       â””â”€â”€ fct_cohorts.sql       # table: weekly acquisition cohorts
â”‚   â””â”€â”€ semantic/
â”‚       â””â”€â”€ semantic_layer.yaml      # governed registry: 47 metrics / 19 dims (MetricFlow)
â”œâ”€â”€ snapshots/
â”‚   â””â”€â”€ snap_dim_items.sql           # SCD2 on item price/category
â”œâ”€â”€ tests/
â”‚   â”œâ”€â”€ assert_session_conversion_rate_bounds.sql  # singular: 0 ≤ rate ≤ 1
â”‚   â””â”€â”€ assert_funnel_monotonicity.sql             # singular: reached_* monotone
â”œâ”€â”€ analyses/
â”‚   â””â”€â”€ adhoc_mix_shift_explore.sql   # compiled-only exploration, never materialized
â”œâ”€â”€ exposures/
â”‚   â””â”€â”€ _exposures.yml               # 7 agents + the Decision Brief
â””â”€â”€ .github/
    â””â”€â”€ workflows/
        â”œâ”€â”€ ci.yml                  # slim CI: state:modified+ --defer + dbt test
        â””â”€â”€ prod_build.yml          # merge-to-main prod refresh + source freshness gate
```

The split of YAML config files (`_ga4__sources.yml` , `_ga4__models.yml` , `_core__models.yml` , ...) follows the dbt convention of one config file per folder, prefixed with `_` so it sorts to the top. Exposures and sources are deliberately separated from model `schema.yml` files to keep each file

single-purpose.

dbt_project.yml

Folder-level `+config` blocks set materialization, partitioning, clustering, grouping/access, tags, and `persist_docs` once per layer, so individual model files carry only the overrides they actually need. This is where the BigQuery cost controls and the access boundary between layers are declared.

```yaml
name: helios
version: '1.0.0'
config-version: 2
require-dbt-version: '≥1.7.0'

profile: helios

model-paths: ['models']
seed-paths: ['seeds']
macro-paths: ['macros']
snapshot-paths: ['snapshots']
test-paths: ['tests']
analysis-paths: ['analyses']

clean-targets: ['target', 'dbt_packages']

vars:
  # Build window â€" matches the static public sample; prod overrides via --vars.
  ga4_start_date: '2020-11-01'
  ga4_end_date:   '2021-01-31'
  # Late-arrival lookback for incremental insert_overwrite.
  incremental_lookback_days: 3
  # Engaged-session threshold (msec); keeps the rule in one place.
  engagement_time_threshold_msec: 10000
  # dbt_project_evaluator scoping
  'dbt_project_evaluator:exclude_packages': true

# Surrogate-key salt + dispatch so dbt_utils macros resolve on BigQuery first.
dispatch:
  - macro_namespace: dbt_utils
    search_order: ['helios', 'dbt_utils']

models:
  +persist_docs:
    relation: true
    columns: true

  helios:
    staging:
      +materialized: view
      +group: staging
      +access: private              # nothing outside staging may ref a stg_ model
      +tags: ['staging', 'ga4']
      +schema: staging

    intermediate:
      +materialized: ephemeral       # inlined; never exposed to BI
      +group: intermediate
      +access: private
      +tags: ['intermediate', 'ga4']

    marts:
      +schema: marts
      core:
        +materialized: incremental
        +incremental_strategy: insert_overwrite
        +partition_by:
          field: event_date
          data_type: date
          granularity: day
        +cluster_by: ['device_category', 'channel_group']
        +require_partition_filter: true
        +on_schema_change: append_new_columns
        +group: marts
        +access: public              # the semantic layer + exposures consume these
        +tags: ['marts', 'core']
        # Dims are small + non-partitioned: override to table.
        dim_users:    { +materialized: table, +require_partition_filter: false }
        dim_items:    { +materialized: table, +require_partition_filter: false }
```

```
        dim_channels: { +materialized: table, +require_partition_filter: false }
        dim_date:     { +materialized: table, +require_partition_filter: false }

      finance:
        +materialized: table
        +group: marts
        +access: public
        +tags: ['marts', 'finance']

      growth:
        +materialized: table
        +group: marts
        +access: public
        +tags: ['marts', 'growth']

    semantic:
      +materialized: view
      +group: marts
      +access: public
      +tags: ['semantic']

seeds:
  helios:
    channel_group_mapping:
      +column_types:
        source: string
        medium: string
        channel_group: string

snapshots:
  helios:
    +target_schema: snapshots
    +tags: ['snapshot']
```

Two things are load-bearing here. First, `+access: private` on staging and intermediate is enforced by dbt's **groups/access** feature: any attempt to `ref('stg_ga4__events')` from a mart in a different group fails compilation, structurally preventing the "no layer reaches across or upward" rule from being violated. Marts are `public` so the semantic layer and exposures can consume them. Second, `require_partition_filter: true` on the partitioned core facts forces every downstream query (including ad-hoc ones the warehouse-mcp issues) to supply an `event_date` predicate, which is the primary defense against a full-table scan blowing the byte budget. The four dims override `require_partition_filter` back to `false` because they are unpartitioned tables.

packages.yml

```
packages:
  - package: dbt-labs/dbt_utils
    version: ["≥1.1.0", "<2.0.0"]
  - package: calogica/dbt_expectations
    version: ["≥0.10.0", "<0.11.0"]
  - package: dbt-labs/dbt_project_evaluator
    version: ["≥0.14.0", "<0.15.0"]
  - package: dbt-labs/codegen
    version: ["≥0.12.0", "<0.13.0"]
```

- **dbt_utils** supplies `generate_surrogate_key` (used for `session_key` callers that need a deterministic composite key), `accepted_range`, `equal_rowcount`, and `unique_combination_of_columns` (the test that proves the grain of `fct_daily_funnel` is genuinely `day x channel_group x device_category x country x is_new_user`).
- **dbt_expectations** adds distributional and statistical tests (`expect_column_values_to_be_between`, `expect_column_values_to_not_be_null`, `expect_row_values_to_have_recent_data`) that data-layer tests rely on for rate bounds and freshness.
- **dbt_project_evaluator** is run in CI to enforce structural conventions automatically: it flags a mart that refs a source directly, a model with no description, a staging model not named `stg_*`, or a fan-out join â€" catching layering violations the access groups can't.
- **codegen** generates boilerplate (`generate_source`, `generate_model_yaml`) so new staging columns are scaffolded, not hand-typed.

After cloning, `dbt deps` installs these into `dbt_packages/`. CI runs `dbt deps` before every build.

profiles.yml

Two targets, two BigQuery datasets, two auth methods. Developers authenticate interactively with OAuth (no keys on laptops); production authenticates via **Workload Identity Federation** from GitHub Actions, so there are no long-lived service-account JSON keys anywhere in the system.

```yaml
helios:
  target: dev
  outputs:
    dev:
      type: bigquery
      method: oauth                 # gcloud ADC; no key files on developer machines
      project: helios-analytics
      dataset: helios_dev           # personal/shared dev marts
      location: US
      threads: 8
      priority: interactive
      job_execution_timeout_seconds: 600
      job_retries: 1
      maximum_bytes_billed: 5368709120   # 5 GiB hard cap per query (the run byte budget)

    prod:
      type: bigquery
      method: oauth                 # WIF-issued ADC token in CI; no JSON key
      project: helios-analytics
      dataset: helios_prod
      location: US
      threads: 16
      priority: batch               # cheaper, non-interactive for scheduled refresh
      job_execution_timeout_seconds: 1800
      job_retries: 2
      maximum_bytes_billed: 21474836480  # 20 GiB ceiling for the full prod refresh
```

`location: US` is mandatory and must match `bigquery-public-data` (the GA4 sample lives in the US multi-region); a mismatch causes "dataset not found in location" errors. `priority: interactive` in dev gives developers fast feedback; `priority: batch` in prod queues jobs against idle slots to lower cost. `maximum_bytes_billed` is set per target â€” 5 GiB in dev maps directly to the project's per-run byte budget so a careless dev query fails fast rather than silently costing money.

In CI, `method: oauth` consumes a short-lived access token minted by the `google-github-actions/auth` step using WIF; dbt reads it from Application Default Credentials. No `keyfile` path appears anywhere.

Conventions & naming

These are the rules that make the model catalog self-consistent. They are enforced by a mix of `dbt_project.yml` config, `dbt_project_evaluator`, and code review.

- **Layer prefixes.** `stg_<source>__<entity>` for staging, `int_<source>__<entity>` for intermediate, `fct_*` / `dim_*` for marts. The source group is `src_ga4`. Everything is `snake_case` â€” files, models, columns, tests.
- **One model per file.** A `.sql` file produces exactly one relation, named identically to the file. No model defines two `SELECT`s; reusable logic lives in macros or an intermediate model.
- **No cross-layer or upward refs.** A model may only `ref()` models in the layer immediately upstream (or the same layer for intermediate fan-in). Marts never `ref()` a source â€” they go through staging. This is enforced structurally by `+access: private` on staging/intermediate and audited by `dbt_project_evaluator`'s `fct_rejoining_of_upstream_concepts` / direct-source-dependency checks.
- **session_key.** The one canonical expression, emitted by the `sessionize()` macro and used nowhere else by hand:

  ```sql
  session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))
  ```

  Session counts are always `COUNT(DISTINCT session_key)` â€” never `FARM_FINGERPRINT`, never `COUNT(*)`.
- *reached_ funnel flags are MAX-DOWNSTREAM (monotonic).* `reached_add_to_cart` is true if the session fired `add_to_cart` *or any later stage*. This guarantees `sessions ≥ reached_view_item ≥ reached_add_to_cart ≥ … ≥ reached_purchase`, so every step rate is `≤ 1` by construction. The retired `did_*` naming is forbidden. `assert_funnel_monotonicity` enforces this at build time.
- **engaged_session** = `session_engaged = '1' OR engagement_time_msec ≥ var('engagement_time_threshold_msec')` (10000) â€” the threshold lives in `vars`, defined once.
- **Channel grouping lives in exactly one macro.** `channel_group_case()` (in `macros/channel_group.sql`) is the single source of truth for the 10 groups â€” Direct, Organic Search, Paid Search, Display, Paid Social, Organic Social, Email, Affiliates, Referral, Other. It is `has_gclid`-aware and backed by `seeds/channel_group_mapping.csv`. No `CASE` statement that buckets channels may exist anywhere else; an 11th group is a hard error.
- **Money & rates.** Only `*_in_usd` columns are aggregated. Rates are computed as `SUM(numerator)/SUM(denominator)` *after* grouping â€” never an average of per-segment ratios (this is the Simpson's-paradox defense). The `traffic_source` gotcha: event-level `traffic_source.*` is user first-touch, not session source â€” prefer session-scoped `event_params` source/medium and fall back to `traffic_source` only when null.

- **Marts are wide.** Each fact carries its descriptive dimensions denormalized (device, channel, geo, date) so the semantic layer slices without runtime joins.

Environments

Two physical BigQuery datasets isolate work, selected by dbt target in `profiles.yml`:

Concern	dev	prod
dataset	`helios_dev`	`helios_prod`
auth	OAuth / gcloud ADC	WIF-minted ADC (no keys)
priority	interactive	batch
`maximum_bytes_billed`	5 GiB	20 GiB
build window (`vars`)	full static sample	overridden to live window

Alongside the two dbt targets, the platform reads/writes the `marts`, `semantic`, `helios_memory`, and `helios_eval` schemas; the semantic layer (`semantic_layer.yaml`) is identical across environments so the agents behave consistently dev-to-prod.

Defer to prod. Slim CI builds only changed models and their children with `--defer --state ./prod-manifest`, resolving unbuilt upstream `ref()` s against the production relations. A developer can build a single mart in `helios_dev` while its (unchanged) upstream staging models resolve to `helios_prod`, so no one has to materialize the entire DAG to test one model.

Blue-green / Write-Audit-Publish. At the session/day grain Helios operates, partition-atomic `insert_overwrite` already gives a clean swap â€" a failed incremental run leaves prior partitions untouched and the marts queryable. Full blue-green or a WAP staging dataset is therefore not warranted here; we note it as the upgrade path if Helios ever serves a live external dashboard where a partial mart must never be visible. For now, `insert_overwrite` idempotency plus the source-freshness gate (Section 2) is the correctness boundary.

---

2. Source Models

A dbt project must never `ref()` or hard-code a raw warehouse table. Instead it **declares sources** in a `sources.yml` file, then references them with `source('src_ga4', 'events')`. This gives three things for free: a single place to repoint the warehouse (public sample today, your own live GA4 export tomorrow), source-level freshness SLAs that can gate the build, and source documentation that flows into the generated docs site and lineage graph. For Helios, the source is the date-sharded GA4 export, and the source declaration is the contract that fails loudly the day Google changes the export schema.

sources.yml for src_ga4

```yaml
# models/staging/_ga4__sources.yml
version: 2

sources:
  - name: src_ga4
    description: >
      GA4 BigQuery Export (Google Analytics 4 e-commerce events). In production this
      is the project's own live daily export dataset; on the public sample it is the
      static, obfuscated Google Merchandise Store export covering 2020-11-01..2021-01-31.
    database: bigquery-public-data                # prod: your own GCP project
    schema: ga4_obfuscated_sample_ecommerce       # prod: analytics_<property_id>
    loader: GA4 BigQuery Export
    loaded_at_field: parse_date('%Y%m%d', event_date)

    # PRODUCTION freshness SLA â€" informational only on the static sample (see note).
    freshness:
      warn_after:  { count: 36, period: hour }
      error_after: { count: 48, period: hour }

    tables:
      - name: events
        description: >
          Date-sharded GA4 event stream. One physical table per day,
          events_YYYYMMDD; the wildcard identifier unions them. Grain = one row
          per event per user per session. Carries nested event_params, items,
          device, geo, traffic_source, and ecommerce STRUCT/ARRAY fields.
        identifier: 'events_*'                    # wildcard over the date shards
```

```yaml
      columns:
        - name: event_date
          description: 'Shard date as YYYYMMDD string; also exposed via _TABLE_SUFFIX.'
          tests:
            - not_null
        - name: event_timestamp
          description: 'Event time, microseconds since UNIX epoch (INT64).'
          tests:
            - not_null
        - name: event_name
          description: 'GA4 event name (e.g. session_start, view_item, purchase).'
          tests:
            - not_null
        - name: user_pseudo_id
          description: 'Cookie/device-grain pseudonymous user id; basis of session_key.'
          tests:
            - not_null
        - name: ecommerce.transaction_id
          description: 'Purchase transaction id; null for non-purchase events.'

      # Loud-failure contract: alert if the raw event volume collapses.
      tests:
        - dbt_utils.recency:
            datepart: day
            field: parse_date('%Y%m%d', event_date)
            interval: 2
            config:
              severity: warn        # warn on the static sample; error in prod overlay
```

The source declaration carries the schema contract (the columns the staging layer depends on are named and `not_null`-tested) and the freshness SLA. Staging models reference it as `{{ source('src_ga4', 'events') }}`; no staging model contains a literal `bigquery-public-data....` reference.

Declaring sources, never raw refs

The rule is absolute: **staging is the only layer allowed to touch the source, and it does so only through `source()`**. Concretely, `stg_ga4__events` begins:

```sql
-- models/staging/stg_ga4__events.sql
with source as (
    select *
    from {{ source('src_ga4', 'events') }}
    where _table_suffix between
          replace('{{ var("ga4_start_date") }}', '-', '')
      and replace('{{ var("ga4_end_date") }}',   '-', '')
)
...
```

This single indirection is what lets the same dbt code run against the public sample in dev and a real GA4 export in prod by changing only `database` / `schema` in `sources.yml`. It also makes the lineage graph correct: `dbt docs` shows `src_ga4.events → stg_ga4__events → …` and `dbt_project_evaluator` can flag any model that skips the source.

Date-sharded wildcard + _TABLE_SUFFIX pruning

GA4 exports one physical table per day, `events_YYYYMMDD`. The `identifier: 'events_*'` wildcard tells BigQuery to union them, and the pseudo-column `_TABLE_SUFFIX` (the part matched by `*`, e.g. `20210114`) is the **partition-pruning lever**. Filtering on `_TABLE_SUFFIX` makes BigQuery scan only the matching shards â€” the single biggest cost control in the project, because an unfiltered `events_*` scans the entire history.

```sql
-- prune shards: scan only the build window (dev) or the incremental lookback (prod)
where _table_suffix between '20201101' and '20210131'
```

On incremental prod runs the predicate narrows further to the lookback window so only newly landed shards are scanned:

```sql
{% if is_incremental() %}
  -- only reprocess the last N days of shards (late-arrival window)
  where _table_suffix ≥ format_date(
          '%Y%m%d',
          date_sub(_dbt_max_partition, interval {{ var('incremental_lookback_days') }} day)
        )
{% endif %}
```

Combined with `require_partition_filter` on the downstream facts and `maximum_bytes_billed`, this keeps a full pipeline run inside the 5 GiB-per-query budget. We never `SELECT *` into a mart; staging selects an explicit column list so unused nested fields aren't scanned.

Source documentation

Descriptions on the source, table, and columns (above) render into the dbt docs site and the lineage DAG, and `persist_docs` pushes the table/column descriptions into BigQuery's own metadata so they appear in the BigQuery console too. This is the human-readable contract: when a developer or agent asks "what is `user_pseudo_id`?", the answer lives next to the declaration, not in tribal knowledge. Doc blocks in `_ga4__docs.md` (e.g. a `session_key` definition) are referenced with `{{ doc('session_key') }}` so the explanation is written once and reused across every layer that surfaces the column.

Source freshness and how it gates the build

`loaded_at_field` is `parse_date('%Y%m%d', event_date)` â€” the freshest event date becomes the table's "load time." `dbt source freshness` compares the max `event_date` against `warn_after` (36h) / `error_after` (48h) and writes a `sources.json` result:

```
# Run freshness checks against the declared SLA
dbt source freshness

# Gate the production build on it: only build if the source is fresh.
dbt source freshness && dbt build --target prod
```

In the scheduled prod pipeline, `dbt source freshness` runs first and its exit code gates everything downstream. If the latest GA4 shard is older than 48 hours the command exits non-zero, the build is aborted, and â€” critically â€” the agent pipeline does **not** run. A stale source must block diagnosis rather than let Monitor/Decompose silently reason over yesterday's (or last week's) data. The freshness result also feeds the slim-CI selector `source_status:fresher+`, which can restrict a run to only the models downstream of sources that have new data.

Production note (honesty about the static sample)

The pinned freshness thresholds describe the **production** strategy: in prod, `src_ga4` points at the project's own GA4 export dataset (`analytics_<property_id>`), which lands a new `events_YYYYMMDD` shard daily, and a 36h/48h SLA correctly catches a stalled export.

On the **public sample**, `bigquery-public-data.ga4_obfuscated_sample_ecommerce` is static and historical â€” the newest shard is `events_20210131` and it never updates. Real freshness against "now" (2026) is therefore meaningless: every freshness check would scream `error`. So on the sample the freshness declaration is **informational only**, and the project degrades gracefully:

- The `dbt_utils.recency` source test is set to `severity: warn` (not `error`) so a static sample doesn't hard-fail every build.
- The prod pipeline's `dbt source freshness && dbt build` gate is enabled only on the `prod` target; the dev/sample build skips the gate (or runs `dbt source freshness` non-blocking, purely to exercise the code path).
- Documentation and tests still validate the *shape* of freshness â€” that `loaded_at_field` parses, that the SLA YAML is well-formed â€” so the moment Helios is repointed at a live export, the gate is already wired and correct with no code change beyond `sources.yml`'s `database` / `schema`.

This keeps the guide honest: the freshness machinery is real, production-grade, and fully implemented; on the frozen sample it is intentionally non-blocking because there is no "fresh" to measure.

3. Staging Models

Staging is the **renaming layer**, and nothing more. Each staging model is a thin, mechanical, **1:1 projection of exactly one source object** whose only jobs are: rename to `snake_case`, cast to the right type, parse the shard suffix into a real `event_date`, and perform *light* flattening of GA4's nested-but-scalar structs (`device.*`, `geo.*`, `ecommerce.*`). The discipline that makes the rest of the project tractable is the list of things staging **must not** do:

- **No joins.** A staging model touches one source and only that source. The moment you join, you have business logic, and business logic lives downstream.
- **No aggregations.** Staging preserves the source grain. `stg_ga4__events` is one row per event; `stg_ga4__event_params` is one row per `(event, param key)`. No `GROUP BY`, no window collapsing.
- **No deduplication or filtering of business rows.** Staging keeps every event. Sessionization, funnel logic, and "engaged session" rules are intermediate concerns.
- **No `SELECT *`.** Every column is named explicitly. This is a BigQuery cost rule (column pruning) *and* a contract: a new column in the GA4 export can never silently leak into marts.

Because staging is pure projection it is materialized as a `view` â€" zero storage cost, always reflects the latest source, and the renamed/typed shape is computed at query time. The two staging models are the *only* place in the entire project that references the raw `event_params` `ARRAY<STRUCT>` and the GA4 nested structs directly. Everything downstream consumes the clean, typed columns and the long param table, so we never re-implement `UNNEST` or remember which key is an int vs. a string twice.

3.1 Source declaration (`src_ga4`)

Staging reads through `source('src_ga4', 'events')`, never a hard-coded table name. The source pins the date-sharded `events_*` wildcard, the partition expectation, and **freshness**. Honesty note: the public `bigquery-public-data.ga4_obfuscated_sample_ecommerce` sample is **static and historical** (2020-11-01..2021-01-31, no live updates), so a real freshness check is **N/A** on it â€" `dbt source freshness` will always report the data as stale because the newest shard is years old. We therefore design the **production** freshness contract (a live daily GA4 export should land `events_YYYYMMDD` within ~36h) and **disable the freshness error on the sample** so the build is honest rather than red for a reason that doesn't apply.

```yaml
# models/staging/src_ga4.yml
version: 2

sources:
  - name: src_ga4
    database: bigquery-public-data
    schema: ga4_obfuscated_sample_ecommerce
    description: >
      Raw GA4 BigQuery export (date-sharded events_YYYYMMDD). The PUBLIC SAMPLE
      is static/historical (2020-11-01..2021-01-31); in PRODUCTION this is the
      live daily export landing one shard per day.
    # PRODUCTION freshness SLA. On the static sample this always trips because
    # the newest shard is historical, so we override loaded_at_field below and
    # rely on a date-bounded singular test instead of dbt source freshness.
    freshness:
      warn_after: {count: 36, period: hour}
      error_after: {count: 48, period: hour}
    # _TABLE_SUFFIX is 'YYYYMMDD'; parse it to a timestamp for freshness.
    loaded_at_field: "parse_timestamp('%Y%m%d', _table_suffix)"
    tables:
      - name: events
        identifier: "events_*"   # wildcard over the date shards
        description: One row per GA4 event; partitioned/sharded by event date.
        # On the sample, override to skip freshness (data is intentionally old):
        # freshness: null
        loaded_at_field: "parse_timestamp('%Y%m%d', _table_suffix)"
```

3.2 `stg_ga4__events.sql`

One row per event, typed and renamed, with the scalar session/device/geo/ecommerce fields surfaced and `session_key` computed once via the `sessionize()` macro. This is where `_TABLE_SUFFIX` becomes a real `event_date` (the partition column for every downstream incremental fact), and where the `is_incremental()` lookback prunes shards so a daily run scans three shards, not ninety.

```sql
-- models/staging/stg_ga4__events.sql
{{ config(materialized='view') }}

with source as (

    select
        -- shard suffix 'YYYYMMDD' → a real DATE partition key
        parse_date('%Y%m%d', _table_suffix)                as event_date,
        _table_suffix                                      as table_suffix,

        -- identity & event
        event_name                                         as event_name,
        timestamp_micros(event_timestamp)                  as event_timestamp,
        user_pseudo_id                                     as user_pseudo_id,
        user_id                                            as user_id,          -- null on this cookie-grain sample

        -- session id lives in event_params, not a top-level column
        {{ get_event_param('ga_session_id',     'int') }}    as ga_session_id,
        {{ get_event_param('ga_session_number', 'int') }}    as ga_session_number,
        {{ get_event_param('page_location',     'string') }} as page_location,
        {{ get_event_param('session_engaged',   'string') }} as session_engaged,
        {{ get_event_param('engagement_time_msec','int') }} as engagement_time_msec,

        -- SESSION-SCOPED source/medium/campaign (see traffic_source gotcha in 4.1)
        {{ get_event_param('source',   'string') }}          as event_source,
```

```
        {{ get_event_param('medium',   'string') }}          as event_medium,
        {{ get_event_param('campaign', 'string') }}          as event_campaign,
        {{ get_event_param('gclid',    'string') }}          as gclid,

        -- USER first-touch attribution (FALLBACK only)
        traffic_source.source                                as first_touch_source,
        traffic_source.medium                                as first_touch_medium,
        traffic_source.name                                  as first_touch_campaign,

        -- device.* (light struct flattening; no joins)
        device.category                                      as device_category,
        device.operating_system                              as operating_system,
        device.web_info.browser                              as browser,
        device.language                                      as device_language,

        -- geo.*
        geo.country                                          as country,
        geo.region                                           as region,
        geo.city                                             as city,

        -- ecommerce.* (purchase rows; usd only per money convention)
        ecommerce.transaction_id                             as transaction_id,
        ecommerce.purchase_revenue_in_usd                    as purchase_revenue_in_usd,
        ecommerce.refund_value_in_usd                        as refund_value_in_usd,
        ecommerce.shipping_value_in_usd                      as shipping_value_in_usd,
        ecommerce.tax_value_in_usd                           as tax_value_in_usd,
        ecommerce.total_item_quantity                        as total_item_quantity

    from {{ source('src_ga4', 'events') }}

    where 1 = 1
    -- PRODUCTION incremental note: although this view is not itself incremental,
    -- it is the upstream of incremental facts. Downstream incrementals prune the
    -- shard scan with a 3-day lookback; to bound the view's own scan you can add:
    --   {% if target.name == 'prod' %}
    --     and parse_date('%Y%m%d', _table_suffix)
    --           between date '2020-11-01' and date '2021-01-31'   -- sample window
    --   {% endif %}
    -- On a LIVE export, replace the fixed window with a relative bound, e.g.
    --   and _table_suffix ≥ format_date('%Y%m%d', date_sub(current_date(), interval 3 day))
    -- so a daily run scans ~3 shards. Prune via _TABLE_SUFFIX, never SELECT *.

),

final as (

    select
        {{ sessionize() }}                                   as session_key,
        *
    from source

)

select * from final
```

`session_key` is built **only** through `sessionize()` so the canonical expression `TO_HEX(MD5(CONCAT(user_pseudo_id,'-',CAST(ga_session_id AS STRING))))` exists in exactly one place. The `select *` in `final` is over an already-explicit CTE (every column was named in `source`), so the no-`SELECT *`-against-source rule still holds.

3.3 `stg_ga4__event_params.sql`

GA4 stores `event_params` as an `ARRAY<STRUCT<key STRING, value STRUCT< … >>>`. Re-`UNNEST`-ing it in every model that needs a param is both error-prone and expensive. This staging model flattens it **once** into a long key/value table at `(event x param key)` grain so downstream code can `where key = '…'` instead of writing `UNNEST` and remembering which of `string_value` / `int_value` / `float_value` / `double_value` is populated.

```
-- models/staging/stg_ga4__event_params.sql
{{ config(materialized='view') }}

with events as (

    select
        parse_date('%Y%m%d', _table_suffix)        as event_date,
        event_name,
        event_timestamp,
        user_pseudo_id,
```

```
        event_params                                -- the raw ARRAY<STRUCT> column
    from {{ source('src_ga4', 'events') }}

),

unnested as (

    select
        e.event_date,
        e.event_name,
        timestamp_micros(e.event_timestamp)         as event_timestamp,
        e.user_pseudo_id,
        ep.key                                      as param_key,
        ep.value.string_value                       as string_value,
        ep.value.int_value                          as int_value,
        ep.value.float_value                        as float_value,
        ep.value.double_value                       as double_value,
        -- a single coalesced text projection for convenience downstream
        coalesce(
            ep.value.string_value,
            cast(ep.value.int_value    as string),
            cast(ep.value.float_value  as string),
            cast(ep.value.double_value as string)
        )                                           as value_string
    from events e,
         unnest(e.event_params) as ep

)

select * from unnested
```

3.4 The three macros

These are the only abstractions staging and intermediate share. Each is a **single source of truth**.

`get_event_param(key, type)` â€" typed extraction from the `event_params` array. It generates the `(SELECT ... FROM UNNEST(event_params) WHERE key = ...)` correlated subquery and selects the right value slot for the requested type, so models never hand-write `UNNEST` for a single key.

```
-- macros/get_event_param.sql
{% macro get_event_param(key, type='string') %}
    {%- set slot = {
        'string': 'string_value',
        'int':    'int_value',
        'float':  'float_value',
        'double': 'double_value'
    } -%}
    (
        select ep.value.{{ slot[type] }}
        from unnest(event_params) as ep
        where ep.key = '{{ key }}'
        limit 1
    )
{% endmacro %}
```

`sessionize()` â€" the canonical session-key expression, defined once. `sessions = COUNT(DISTINCT session_key)` everywhere; the key is never `FARM_FINGERPRINT` and never `COUNT(*)`.

```
-- macros/sessionize.sql
{% macro sessionize(user_col='user_pseudo_id', session_id_col='ga_session_id') %}
    to_hex(md5(concat(
        {{ user_col }}, '-', cast({{ session_id_col }} as string)
    )))
{% endmacro %}
```

`channel_group()` / `channel_group_case()` â€" the **single source of truth** for the 10 GA4 default channel groups. Every model and the semantic layer derive `channel_group` from this macro; there is no 11th group and no "Paid Other". It is `has_gclid`-aware (a `gclid` forces Paid Search even when the medium is dirty) and applies GA4's default-channel-grouping precedence.

```
-- macros/channel_group.sql
{% macro channel_group(source_col='source', medium_col='medium', gclid_col='gclid') %}
  {{ return(channel_group_case(source_col, medium_col, gclid_col)) }}
{% endmacro %}
```

```sql
{% macro channel_group_case(source_col='source', medium_col='medium', gclid_col='gclid') %}
case
    -- Direct: no/(direct)/(none) source and (none)/(not set) medium
    when ( {{ source_col }} is null or lower({{ source_col }}) in ('(direct)','direct') )
         and ( {{ medium_col }} is null or lower({{ medium_col }}) in ('(none)','(not set)') )
        then 'Direct'

    -- Paid Search: gclid present, OR known search source on a paid medium
    when {{ gclid_col }} is not null
         or ( regexp_contains(lower({{ source_col }}), r'google|bing|yahoo|duckduckgo|baidu|yandex|ecosia')
              and regexp_contains(lower({{ medium_col }}), r'^(.*cp.*|ppc|paid|retargeting)$') )
        then 'Paid Search'

    -- Paid Social: known social source on a paid medium
    when regexp_contains(lower({{ source_col }}), r'facebook|instagram|tiktok|twitter|x\.com|linkedin|pinterest|reddit|snapchat')
         and regexp_contains(lower({{ medium_col }}), r'^(.*cp.*|ppc|paid|social-paid|retargeting)$')
        then 'Paid Social'

    -- Display: banner/display/cpm/expandable/interstitial mediums
    when regexp_contains(lower({{ medium_col }}), r'^(display|banner|cpm|expandable|interstitial)$')
        then 'Display'

    -- Organic Search: search engines on organic medium
    when regexp_contains(lower({{ source_col }}), r'google|bing|yahoo|duckduckgo|baidu|yandex|ecosia')
         and lower({{ medium_col }}) = 'organic'
        then 'Organic Search'

    -- Organic Social: social sources, organic/social/referral medium
    when regexp_contains(lower({{ source_col }}), r'facebook|instagram|tiktok|twitter|x\.com|linkedin|pinterest|reddit|snapchat|youtube')
         and lower({{ medium_col }}) in ('social','social-network','social-media','sm','organic','referral')
        then 'Organic Social'

    -- Email
    when lower({{ source_col }}) = 'email'
         or regexp_contains(lower({{ medium_col }}), r'^e?-?mail$')
        then 'Email'

    -- Affiliates
    when lower({{ medium_col }}) = 'affiliate'
        then 'Affiliates'

    -- Referral: any explicit referral medium not caught above
    when lower({{ medium_col }}) in ('referral','link')
        then 'Referral'

    -- Everything else
    else 'Other'
end
{% endmacro %}
```

3.5 Staging `schema.yml`

Staging tests assert the projection didn't break: keys are present, the param key/value pairs are well-formed, and source columns are documented. Heavy reconciliation lives in marts; here we test cheaply and structurally.

```yaml
# models/staging/stg_ga4__schema.yml
version: 2

models:
  - name: stg_ga4__events
    description: "1:1 typed/renamed GA4 events; one row per event. session_key surfaced via sessionize()."
    columns:
      - name: session_key
        description: "TO_HEX(MD5(user_pseudo_id || '-' || ga_session_id)). Null only when ga_session_id is null."
        data_tests:
          - not_null:
              # GA4 emits a few session-less events (e.g. first_open) with null ga_session_id
              config: {where: "ga_session_id is not null"}
      - name: event_date
        description: "Event date parsed from the _TABLE_SUFFIX shard (DATE). Partition key for downstream facts."
        data_tests: [not_null]
      - name: event_timestamp
        description: "Event time (TIMESTAMP, from timestamp_micros)."
        data_tests: [not_null]
      - name: user_pseudo_id
        description: "Device/cookie pseudo id; the de facto user grain (user_id is null on this sample)."
        data_tests: [not_null]
```

```yaml
      - name: ga_session_id
        description: "GA4 session id within a user, from event_params.ga_session_id."
      - name: page_location
        description: "Full page URL of the event, from event_params.page_location."
      - name: event_source
        description: "SESSION-SCOPED source from event_params.source (preferred over first_touch_source)."
      - name: event_medium
        description: "SESSION-SCOPED medium from event_params.medium."
      - name: device_category
        description: "device.category: mobile / desktop / tablet."
      - name: country
        description: "geo.country."
      - name: transaction_id
        description: "ecommerce.transaction_id; non-null only on purchase events."
      - name: purchase_revenue_in_usd
        description: "ecommerce.purchase_revenue_in_usd (USD only, per money convention)."
        data_tests:
          - dbt_utils.accepted_range:
              min_value: 0
              config: {where: "purchase_revenue_in_usd is not null"}

  - name: stg_ga4__event_params
    description: "Fully unnested event_params at (event x param key) grain; the only place UNNEST(event_params) lives."
    columns:
      - name: param_key
        description: "event_params[].key."
        data_tests: [not_null]
      - name: event_timestamp
        description: "Event time (TIMESTAMP)."
        data_tests: [not_null]
      - name: user_pseudo_id
        description: "Device/cookie pseudo id."
        data_tests: [not_null]
      - name: value_string
        description: "Coalesced string projection of the populated value slot."
```

---

4. Intermediate Models

Intermediate is where **business logic** lives â€” the rules that more than one mart needs and that no single mart should own. There are exactly two intermediate models, and both are **keystones**: if they are wrong, every downstream number is *silently* wrong (no error, just a quietly incorrect figure). They are:

- `int_ga4__sessionized` â€” collapses the event stream to **one row per session**, deriving the session-scoped attributes (landing page, source/medium, channel group, device/geo, engagement, new-vs-returning).
- `int_ga4__funnel_steps` â€” collapses to **one row per session** carrying the `reached_*` funnel flags and `session_revenue`.

Intermediate models are **never exposed to BI or the semantic layer directly**. They are plumbing: the semantic layer references only marts, and marts (`fct_sessions` , `fct_funnel` , `fct_daily_funnel` , ...) are built *from* these intermediates. Because they are internal and consumed by exactly the few marts that sit on top of them, they are materialized `ephemeral` (inlined as CTEs into their consumers â€” no storage, no extra scan) or `view` when a model is referenced by several marts and inlining would duplicate the scan. The pinned default is ephemeral.

4.1 `int_ga4__sessionized.sql` (KEYSTONE)

Sessionization groups the event stream by the canonical session key â€” i.e. by `(user_pseudo_id, ga_session_id)` â€” and derives each session's attributes with window/aggregate functions over its events. The two subtle rules:

1. **`landing_page` = the earliest `page_location`.** We take the `page_location` of the event with the minimum `event_timestamp` in the session (`ARRAY_AGG( ... ORDER BY event_timestamp LIMIT 1)`), not just any non-null value.
2. **The `traffic_source` gotcha.** Event-level `traffic_source.*` is GA4's **user first-touch** attribution â€” it is the channel that *originally acquired the user*, identical on every session that user ever has. It is **not** the source of *this* session. Using it would mis-credit returning users' sessions to their original acquisition channel and corrupt every channel-level rate (a Simpson's-paradox-grade error). So we prefer the **session-scoped** `event_params.source/medium` (surfaced as `event_source` / `event_medium` in staging) and **fall back to `traffic_source` only when the session-scoped value is null**.

`is_new_user` is derived from `ga_session_number = 1` (the user's first-ever session), and `engaged_session` from the canonical rule `session_engaged = '1' OR engagement_time_msec ≥ 10000`. The `channel_group` is computed via the macro from the resolved session-scoped source/medium and `gclid` .

```sql
-- models/intermediate/int_ga4__sessionized.sql
{{ config(materialized='ephemeral') }}

with events as (

    select *
    from {{ ref('stg_ga4__events') }}
    where ga_session_id is not null   -- drop session-less events (e.g. first_open)

),

sessionized as (

    select
        session_key,
        user_pseudo_id,
        ga_session_id,

        -- session timing
        min(event_date)                                  as session_date,
        min(event_timestamp)                             as session_start_ts,
        max(event_timestamp)                             as session_end_ts,

        -- landing_page = page_location of the EARLIEST event in the session
        array_agg(page_location ignore nulls
                  order by event_timestamp limit 1)[safe_offset(0)] as landing_page,

        -- SESSION-SCOPED source/medium with traffic_source first-touch FALLBACK.
        -- event_source/event_medium = event_params.* (session scope, preferred).
        -- first_touch_* = traffic_source.* (USER first-touch, fallback only).
        coalesce(
            max(event_source),
            max(first_touch_source)
        )                                                as session_source,
        coalesce(
            max(event_medium),
            max(first_touch_medium)
        )                                                as session_medium,
        coalesce(
            max(event_campaign),
            max(first_touch_campaign)
        )                                                as session_campaign,
        max(gclid)                                       as gclid,

        -- device / geo (stable within a session; max() picks the non-null)
        max(device_category)                             as device_category,
        max(operating_system)                            as operating_system,
        max(browser)                                     as browser,
        max(country)                                     as country,
        max(region)                                      as region,
        max(city)                                        as city,

        -- engagement: canonical rule (≥ , not >)
        max(case
              when session_engaged = '1'
                or engagement_time_msec ≥ 10000 then true
              else false
            end)                                         as engaged_session,

        -- new vs returning: first-ever session for this user
        max(case when ga_session_number = 1 then true else false end) as is_new_user,
        max(ga_session_number)                           as ga_session_number

    from events
    group by session_key, user_pseudo_id, ga_session_id

)

select
    s.*,
    -- channel grouping from the SINGLE SOURCE OF TRUTH macro, on the
    -- RESOLVED session-scoped source/medium (+ gclid awareness)
    {{ channel_group_case('s.session_source', 's.session_medium', 's.gclid') }} as channel_group
from sessionized s
```

4.2 `int_ga4__funnel_steps.sql` (KEYSTONE)

The funnel flags are **MAX-DOWNSTREAM monotonic**: `reached_X` is true if the session fired event `X` **or any later stage**. This guarantees by construction that

```
sessions ≥ reached_view_item ≥ reached_add_to_cart ≥ reached_begin_checkout
        ≥ reached_add_shipping_info ≥ reached_add_payment_info ≥ reached_purchase
```

so every step rate is `≤ 1` and the funnel can never widen as it deepens. (The retired `did_*` flags marked only "the session fired exactly this event" and could be non-monotonic â€" e.g. a session that purchased via a deep link without an explicit `add_to_cart` event would show `did_purchase=1` but `did_add_to_cart=0`, producing a step rate > 1. `reached_*` fixes this.)

Implementation: per event, compute a boolean "this event is step X or downstream of X", then `LOGICAL_OR` it up to the session grain. `session_revenue` is the session's purchase revenue, **deduped** so a session with two `purchase` events (or a duplicated event) is not double-counted â€" we take the max purchase revenue tied to the session's transaction rather than summing event rows.

```sql
-- models/intermediate/int_ga4__funnel_steps.sql
{{ config(materialized='ephemeral') }}

with events as (

    select
        session_key,
        event_name,
        event_date,
        transaction_id,
        purchase_revenue_in_usd
    from {{ ref('stg_ga4__events') }}
    where ga_session_id is not null

),

-- per-event MAX-DOWNSTREAM membership: a 'view_item' event marks reached_view_item;
-- a 'purchase' event marks ALL of reached_view_item..reached_purchase, etc.
flagged as (

    select
        session_key,
        event_date,
        transaction_id,
        purchase_revenue_in_usd,

        event_name in (
            'view_item','add_to_cart','begin_checkout',
            'add_shipping_info','add_payment_info','purchase'
        ) as f_view_item,

        event_name in (
            'add_to_cart','begin_checkout',
            'add_shipping_info','add_payment_info','purchase'
        ) as f_add_to_cart,

        event_name in (
            'begin_checkout','add_shipping_info','add_payment_info','purchase'
        ) as f_begin_checkout,

        event_name in (
            'add_shipping_info','add_payment_info','purchase'
        ) as f_add_shipping_info,

        event_name in (
            'add_payment_info','purchase'
        ) as f_add_payment_info,

        event_name = 'purchase' as f_purchase

    from events

),

session_grain as (

    select
        session_key,
        min(event_date)                                 as session_date,

        logical_or(f_view_item)         as reached_view_item,
        logical_or(f_add_to_cart)       as reached_add_to_cart,
```

```
        logical_or(f_begin_checkout)    as reached_begin_checkout,
        logical_or(f_add_shipping_info) as reached_add_shipping_info,
        logical_or(f_add_payment_info)  as reached_add_payment_info,
        logical_or(f_purchase)          as reached_purchase,

        -- DEDUPED session revenue: one purchase amount per transaction, summed
        -- across distinct transactions in the (rare) multi-order session.
        coalesce(sum(distinct_txn_rev), 0)              as session_revenue,
        min(transaction_id)                             as transaction_id

    from (
        select
            session_key,
            event_date,
            transaction_id,
            f_view_item, f_add_to_cart, f_begin_checkout,
            f_add_shipping_info, f_add_payment_info, f_purchase,
            -- collapse duplicate purchase rows: one revenue value per txn
            max(purchase_revenue_in_usd) over (
                partition by session_key, transaction_id
            ) as distinct_txn_rev
        from flagged
    )
    group by session_key

)

select * from session_grain
```

4.3 Intermediate `schema.yml`

```
# models/intermediate/int_ga4__schema.yml
version: 2

models:
  - name: int_ga4__sessionized
    description: "KEYSTONE. One row per session; session-scoped source/medium (traffic_source fallback), channel_group, engagement, new/returning."
    columns:
      - name: session_key
        data_tests: [not_null, unique]
      - name: channel_group
        description: "From channel_group_case(); one of the 10 canonical groups."
        data_tests:
          - accepted_values:
              values: ['Direct','Organic Search','Paid Search','Display','Paid Social',
                       'Organic Social','Email','Affiliates','Referral','Other']
      - name: engaged_session
        data_tests: [not_null]
      - name: is_new_user
        data_tests: [not_null]

  - name: int_ga4__funnel_steps
    description: "KEYSTONE. One row per session; reached_* MAX-DOWNSTREAM monotonic flags + deduped session_revenue."
    columns:
      - name: session_key
        data_tests: [not_null, unique]
      - name: reached_view_item
        data_tests: [not_null]
      - name: reached_purchase
        data_tests: [not_null]
      - name: session_revenue
        data_tests:
          - dbt_utils.accepted_range: {min_value: 0}
```

4.4 Keystone test budget

Both keystones **fail silently**: a sessionization bug (wrong key, first-touch leakage, a dropped landing page) or a monotonicity bug (a non-downstream flag, a double-counted purchase) produces no error â€” just a number that is quietly wrong and corrupts every mart, every metric, and ultimately every agent diagnosis. Structural tests (`not_null` , `unique` , `accepted_values`) catch shape, not *correctness*. Therefore the keystone test budget is spent **golden-value tests first**: dbt `unit_tests` that seed a handful of synthetic events and assert the *exact* derived output â€” e.g. two events with the same `(user_pseudo_id, ga_session_id)` collapse to one session with the earliest `page_location` as `landing_page` ; a returning user's session takes its `event_params` source/medium, not `traffic_source` ; a session whose only funnel event is `purchase` still sets every `reached_*` flag true (monotonicity); a duplicated `purchase` event yields `session_revenue` counted once. These are backed by the singular tests `assert_funnel_monotonicity` (asserts `sessions ≥ reached_view_item ≥ … ≥ reached_purchase` holds at every grain) and

`assert_session_conversion_rate_bounds` (asserts `purchasing_sessions / sessions` stays within `[0, 1]`). The full keystone testing strategy â€" unit-test `given` / `expect` fixtures, the singular tests, and the reconciliation backbone â€" is specified in **section 6 (Testing)**; this section only flags *that* these two models earn the deepest test investment in the project.

5. Marts

Marts are Helios's **consumption layer** â€" the only models the semantic layer (and therefore the agents) are allowed to read. Everything upstream (`src_ga4` â†' `stg_ga4__*` â†' `int_ga4__*`) exists to feed them; everything downstream (`semantic_layer.yaml` â†' semantic-mcp â†' the 7 agents) consumes them. A mart is the contract between analytics engineering and the product: if a number is in a mart, it is governed, tested, documented, and reconciled against raw. If it is not in a mart, the agents physically cannot reach it.

Marts are organized into three folders â€" **core** (the session/funnel spine + conformed dims), **finance** (orders + line items), and **growth** (decomposition rollups + retention cohorts) â€" matching the `models/marts/{core,finance,growth}/` tree and the per-folder materializations in `dbt_project.yml`.

5.1 Mart catalog

model	folder	layer	grain	primary key	materialization	purpose	feeds grain
`fct_sessions`	core	mart	session	`session_key`	incremental (`insert_overwrite`)	engagement + wide session dims + funnel reach	`fct_s`
`fct_funnel`	core	mart	session	`session_key`	incremental (`insert_overwrite`)	`reached_*` flags + `session_revenue` + wide dims; **primary session grain**	`fct_f`
`fct_daily_funnel`	core	mart	day Ã— [channel_group, device_category, country, is_new_user]	`daily_funnel_key` (md5 of grain)	incremental (`insert_overwrite`)	additive step counts + revenue; Monitor/Decompose/eval feed	(rollup grain)
`dim_users`	core	mart	user_pseudo_id	`user_key`	table	first-touch attrs, new/returning (cookie grain)	dimens `fct_*`
`dim_items`	core	mart	item_id	`item_key`	table	item_name/brand/category(2..5)/price	dimens `fct_o`
`dim_channels`	core	mart	channel_group (10 rows)	`channel_key`	table	the 10 GA4 default channel groups + is_paid/is_organic/order	confor channe
`dim_date`	core	mart	day	`date_key`	table	conformed date spine 2020-11-01..2021-01-31	confor dim
`fct_orders`	finance	mart	transaction_id	`order_key`	table	gross/net/refund/shipping/tax + wide channel/device/date	`fct_o`
`fct_order_items`	finance	mart	transaction_id Ã— item line	`order_item_key`	table	`item_revenue_in_usd`, quantity, item dims	`fct_o`
`fct_funnel_by_dim`	growth	mart	day Ã— canonical dimension	`funnel_by_dim_key`	table	funnel rollup by canonical dims (decomposition input)	(decon input)
`fct_cohorts`	growth	mart	cohort_week Ã— age_days	`cohort_key`	table	weekly acquisition cohorts (day_1/7/30 retention)	`fct_c`

The five base grains the semantic layer resolves against are exactly `fct_funnel`, `fct_sessions`, `fct_orders`, `fct_order_items`, and `fct_cohorts` (see `semantic_layer.yaml`); the other marts are conformed dimensions or pre-aggregated rollups that the agents read indirectly.

5.2 Why marts are WIDE (denormalized) facts

Every Helios fact is **denormalized** â€" it carries its own descriptive dimensions (`device_category`, `channel_group`, `country`, `is_new_user`, `event_date`) physically on the row rather than only foreign keys. This is deliberate and load-bearing for three reasons:

1. **The semantic layer slices without runtime joins.** When the Decompose agent asks `build_query('session_conversion_rate', dims=['device_category','channel_group'])`, semantic-mcp emits a single `GROUP BY` against `fct_funnel` â€" no join graph to plan, no join key cardinality surprises, no fan-out. The mix-vs-rate decomposition is one scan of one table.
2. **BigQuery rewards wide, not normalized.** BigQuery is a columnar scan engine; joins are comparatively expensive and column projection is nearly free. Denormalizing the handful of low-cardinality dims onto each fact costs trivial storage and removes the join from the hot path, which directly serves the **â‰¤5 GiB/run byte budget** and **<5 min time-to-diagnosis** targets.
3. **Determinism and reproducibility.** A wide fact freezes the dimension values as they were at session time. There is no risk of a slowly-changing dimension re-classifying historical rows on the next run â€" the row is self-contained and idempotent under partition overwrite.

Surrogate keys (`session_key`, `order_key`, `channel_key`, `date_key`, `user_key`, `item_key`, `order_item_key`) still exist so the dims can be joined for enrichment and so `relationships` tests can enforce referential integrity, but the agents never need those joins at query time. `dim_channels`, `dim_date`, and `dim_users` are **conformed** â€" the same physical dimension is shared by `fct_sessions`, `fct_funnel`, `fct_orders`, and the daily rollups â€" so a slice by `channel_group` means the same thing in every fact, which is what lets the Decompose agent pivot any metric across the same canonical axes.

5.3 Incremental strategy (core facts)

The three session-grain core facts are large and append-shaped, so they use the production incremental pattern. The other marts (`dim_*`, finance, growth) are small enough to rebuild as `table` every run.

```
{{ config(
    materialized='incremental',
    incremental_strategy='insert_overwrite',
    partition_by={'field': 'event_date', 'data_type': 'date', 'granularity': 'day'},
    cluster_by=['device_category', 'channel_group'],
    require_partition_filter=true,
    on_schema_change='append_new_columns'
) }}
```

- **`insert_overwrite` + `partition_by(event_date)`** replaces *whole day partitions* atomically. A re-run over the same days produces byte-identical output (idempotent); a late-arriving or corrected `events_YYYYMMDD` shard reprocesses cleanly because the entire day is overwritten â€" no surrogate-key dedup, no MERGE, no drift.
- **3-day `is_incremental()` lookback** absorbs GA4's late event landing without reprocessing the full history:

  ```
  {% if is_incremental() %}
    where event_date ≥ date_sub(_dbt_max_partition, interval 3 day)
  {% endif %}
  ```

- **Partition pruning + clustering.** Every diagnosis scopes a date window, so `partition_by(event_date)` prunes to the touched days and `cluster_by(device_category, channel_group)` gives a second block-level filter on the two highest-traffic decomposition dimensions. `require_partition_filter=true` makes a date-less query *error* rather than full-scan.

Honesty on the static sample. The public dataset is frozen at 2020-11-01..2021-01-31, so no new shards ever land and source freshness is effectively N/A. In dev we therefore `dbt build --full-refresh` (3 months rebuilds in seconds and is the safe reset). The incremental config above is the **production** design â€" wired and correct so that the moment Helios points at a live daily GA4 export, the 3-day lookback and partition overwrite work without code changes. The static sample simply never exercises the `is_incremental()` branch.

5.4 Core marts

fct_sessions

- **Grain:** one row per session. **PK:** `session_key`.
- **Key columns:** `session_key`, `user_pseudo_id` (FKâ†' `dim_users`), `event_date` (partition/FKâ†' `dim_date`), `channel_group` (FKâ†' `dim_channels`), `source`, `medium`, `landing_page`, `device_category`, `operating_system`, `browser`, `country`, `region`, `is_new_user`, `ga_session_number`, `event_count`, `engaged_session`, `engagement_time_msec`.
- **Materialization:** incremental `insert_overwrite` â€" large, append-shaped, day-partitioned.
- **Feeds semantic:** base grain `fct_sessions` for volume/engagement metrics (`sessions`, `engaged_sessions`, `engagement_rate`, the RPS denominator).

fct_funnel

- **Grain:** one row per session. **PK:** `session_key`. Joins **1:1** to `fct_sessions`.

- **Key columns:** the wide session dims (same as `fct_sessions`) + the monotonic `reached_*` flags (`reached_view_item` â€¦ `reached_purchase`) + `session_revenue`.
- **Materialization:** incremental `insert_overwrite`.
- **Feeds semantic:** the **primary session grain**. `session_conversion_rate`, `view_to_cart_rate`, all step rates, and the funnel session counts resolve here. Rates are computed as `SUM(numerator)/SUM(denominator)` from the additive `reached_*` flags after grouping â€" never stored, never an average of ratios.

fct_daily_funnel

- **Grain:** one row per (`event_date`, `channel_group`, `device_category`, `country`, `is_new_user`). **PK:** `daily_funnel_key`.
- **Key columns:** additive counts only â€" `sessions`, `view_item_sessions`, `add_to_cart_sessions`, `begin_checkout_sessions`, `add_shipping_info_sessions`, `add_payment_info_sessions`, `purchasing_sessions`, `transactions`, `revenue`.
- **Materialization:** incremental `insert_overwrite` (day-partitioned).
- **Feeds semantic:** the Monitor (time-series anomaly), Decompose (mix-shift), and eval feed. **Rates are NOT stored** â€" only additive counts â€" so they re-aggregate correctly across any slice. It aggregates `fct_funnel` (which carries `session_revenue`), *not* `int_ga4__funnel_steps`; dropping revenue here would break the eval's dollar-at-risk labels.

dim_users

- **Grain:** one row per `user_pseudo_id`. **PK:** `user_key`.
- **Key columns:** `first_touch_ts`, `first_touch_source`, `first_touch_medium`, `first_channel_group`, `total_sessions`, `total_revenue`, `is_purchaser`, `is_new_user`.
- **Materialization:** `table`. **Feeds semantic:** the user-grain dimension/denominator for ARPU; cookie-grain only (`user_id` is null), caveated by the Critic as a cookie approximation.

dim_items, dim_channels, dim_date

- **`dim_items`** â€" PK `item_key`; `item_name`, `item_brand`, `item_category(2..5)`, `price`. Conformed dimension for `fct_order_items`. `table` (with `snap_dim_items` providing SCD2 history on price/category).
- **`dim_channels`** â€" PK `channel_key`; exactly 10 rows, one per `channel_group`, with `is_paid`, `is_organic`, `channel_group_order`. The conformed channel dimension; tested with `accepted_values` of the 10 groups. `table`.
- **`dim_date`** â€" PK `date_key`; conformed date spine 2020-11-01..2021-01-31 with `week`, `month`, `quarter`, `day_of_week`, `is_weekend`. `table`.

5.5 Finance marts

fct_orders

- **Grain:** one row per `transaction_id` (deduped). **PK:** `order_key` = `transaction_id`.
- **Key columns:** `session_key` (FKâ†' `fct_sessions`), `user_pseudo_id` (FKâ†' `dim_users`), `event_date` (FKâ†' `dim_date`), `channel_group` (FKâ†' `dim_channels`), `device_category`, `order_ts`, `gross_revenue`, `refund_value_in_usd`, `net_revenue`, `shipping_value_in_usd`, `tax_value_in_usd`, `total_item_quantity`, `unique_items`.
- **Materialization:** `table` (small; full rebuild is cheap). **Feeds semantic:** base grain `fct_orders` for `revenue`, `gross_revenue`, `net_revenue`, `transactions`, `aov`, `items_per_transaction`.

fct_order_items

- **Grain:** one row per (`transaction_id`, item line). **PK:** `order_item_key` = md5(transaction_id + item_id + line ordinal).
- **Key columns:** `order_key` (FKâ†' `fct_orders`), `item_key` (FKâ†' `dim_items`), `item_name`, `item_category`, `quantity`, `item_revenue_in_usd`, `price_in_usd`, `coupon`.
- **Materialization:** `table`. **Feeds semantic:** base grain `fct_order_items` for item-revenue and category diagnoses.

5.6 Growth marts

fct_funnel_by_dim

- **Grain:** one row per (`event_date`, canonical dimension name/value). **PK:** `funnel_by_dim_key`.
- **Purpose:** a long-format funnel rollup keyed by `dimension_name` + `dimension_value` so the Decompose agent's mix-vs-rate decomposition can iterate over canonical dimensions uniformly. `table`. (Decomposition input; not a semantic base grain.)

fct_cohorts

- **Grain:** one row per (`cohort_week` , `age_days`). **PK:** `cohort_key` .
- **Key columns:** `cohort_week` , `age_days` , `cohort_size` , `active_users` , `retention_rate` .
- **Materialization:** `table` . **Feeds semantic:** base grain `fct_cohorts` for `day_1/7/30_retention` . Retention denominators are the fixed original cohort size (never the surviving population), so retention is monotonically non-increasing.

5.7 Marts best practices applied here

- **One mart per business entity.** `fct_funnel` =session, `fct_orders` =transaction, `fct_order_items` =order line, `fct_cohorts` =cohortÃ—age. No mart mixes grains.
- **No BI-only logic in marts.** Rates, ratios, and derived metrics (`session_conversion_rate` , `aov` , RPS) live **only** in `semantic_layer.yaml` , computed `SUM(num)/SUM(den)` after grouping. Marts store additive primitives; presentation/derivation is the semantic layer's job.
- **Business logic lives upstream, exactly once.** Sessionization and `reached_*` flags live in the `int_ga4__*` keystones; channel grouping lives only in `channel_group_case()` . Marts assemble, they do not re-derive.
- **Mart â†' semantic grain mapping** is explicit and 1:1: `fct_funnel` , `fct_sessions` , `fct_orders` , `fct_order_items` , `fct_cohorts` are the only base grains the semantic layer resolves against.

5.8 fct_funnel.sql

```
-- models/marts/core/fct_funnel.sql
-- Session grain (PK session_key). Joins int_ga4__sessionized + int_ga4__funnel_steps 1:1.
-- Wide session dims + monotonic reached_* flags + session_revenue.
{{ config(
    materialized='incremental',
    incremental_strategy='insert_overwrite',
    partition_by={'field': 'event_date', 'data_type': 'date', 'granularity': 'day'},
    cluster_by=['device_category', 'channel_group'],
    require_partition_filter=true,
    on_schema_change='append_new_columns'
) }}

with sess as (
    select * from {{ ref('int_ga4__sessionized') }}
    {% if is_incremental() %}
      where date(timestamp_micros(session_start_micros))
            ≥ date_sub(_dbt_max_partition, interval 3 day)
    {% endif %}
),

steps as (
    select * from {{ ref('int_ga4__funnel_steps') }}
)

select
    -- keys / partition
    s.session_key,
    s.user_pseudo_id,
    date(timestamp_micros(s.session_start_micros))      as event_date,   -- partition col
    -- wide, denormalized session dimensions (no runtime join needed downstream)
    s.channel_group,
    s.source,
    s.medium,
    s.landing_page,
    s.device_category,
    s.operating_system,
    s.browser,
    s.country,
    s.region,
    (s.ga_session_number = 1)                           as is_new_user,
    s.engaged_session,
    -- monotonic, max-downstream funnel flags (sessions ≥ reached_view_item ≥ ... ≥ reached_purchase)
    st.did_session_start,
    st.reached_view_item,
    st.reached_add_to_cart,
    st.reached_begin_checkout,
    st.reached_add_shipping_info,
    st.reached_add_payment_info,
    st.reached_purchase,
    -- per-session deduped revenue (one purchase_revenue per distinct transaction_id)
    coalesce(st.session_revenue, 0.0)                   as session_revenue
from sess s
join steps st using (session_key)   -- 1:1; both are session-grained on session_key
```

5.9 fct_daily_funnel.sql

```
-- models/marts/core/fct_daily_funnel.sql
-- Additive daily funnel: COUNT(DISTINCT IF(reached_*, session_key, NULL)) per step + revenue.
-- Grouped by event_date x canonical dims. RATES ARE NOT STORED (semantic layer computes them).
-- Aggregates fct_funnel (carries session_revenue) -- NOT int_ga4__funnel_steps.
{{ config(
    materialized='incremental',
    incremental_strategy='insert_overwrite',
    partition_by={'field': 'event_date', 'data_type': 'date', 'granularity': 'day'},
    require_partition_filter=true
) }}

with funnel as (
    select * from {{ ref('fct_funnel') }}
    {% if is_incremental() %}
      where event_date ≥ date_sub(_dbt_max_partition, interval 3 day)
    {% endif %}
)

select
    -- surrogate PK over the grain columns
    {{ dbt_utils.generate_surrogate_key(
         ['event_date','channel_group','device_category','country','is_new_user']) }}
                                                                    as daily_funnel_key,
    event_date,
    channel_group,
    device_category,
    country,
    is_new_user,
    -- additive step counts (re-aggregatable across any slice)
    count(distinct session_key)                                     as sessions,
    count(distinct if(reached_view_item,         session_key, null))     as view_item_sessions,
    count(distinct if(reached_add_to_cart,       session_key, null))     as add_to_cart_sessions,
    count(distinct if(reached_begin_checkout,    session_key, null))     as begin_checkout_sessions,
    count(distinct if(reached_add_shipping_info, session_key, null))     as add_shipping_info_sessions,
    count(distinct if(reached_add_payment_info,  session_key, null))     as add_payment_info_sessions,
    count(distinct if(reached_purchase,          session_key, null))     as purchasing_sessions,
    count(distinct if(reached_purchase and session_revenue > 0, session_key, null)) as transactions,
    sum(session_revenue)                                            as revenue
from funnel
group by event_date, channel_group, device_category, country, is_new_user
-- NOTE: session_conversion_rate etc. are NOT selected here. They are derived in
-- semantic_layer.yaml as SUM(purchasing_sessions)/SUM(sessions) after grouping.
```

5.10 fct_orders.sql

```
-- models/marts/finance/fct_orders.sql
-- One deduped row per transaction_id (PK order_key). Wide channel/device/date dims.
-- gross/net/refund/shipping/tax. Small fact → full table rebuild.
{{ config(materialized='table') }}

with purch as (
    -- collapse duplicate purchase rows for one transaction_id (GA4 emits retries/multi-stream)
    select
        ecommerce.transaction_id                            as order_key,
        {{ sessionize() }}                                  as session_key,
        user_pseudo_id,
        timestamp_micros(event_timestamp)                   as order_ts,
        date(timestamp_micros(event_timestamp))             as event_date,
        any_value(ecommerce.purchase_revenue_in_usd)        as gross_revenue,
        any_value(ecommerce.refund_value_in_usd)            as refund_value_in_usd,
        any_value(ecommerce.shipping_value_in_usd)          as shipping_value_in_usd,
        any_value(ecommerce.tax_value_in_usd)               as tax_value_in_usd,
        any_value(ecommerce.total_item_quantity)            as total_item_quantity,
        any_value(ecommerce.unique_items)                   as unique_items
    from {{ source('src_ga4','events') }}
    where event_name = 'purchase'
      and ecommerce.transaction_id is not null              -- exclude untagged purchases
      and _table_suffix between '20201101' and '20210131'   -- shard prune the static sample
    group by order_key, session_key, user_pseudo_id, order_ts, event_date
),

dims as (
    -- pull the wide session dims so fct_orders slices without a runtime join
    select session_key, channel_group, device_category, country
    from {{ ref('fct_sessions') }}
```

```
)

select
    p.order_key,
    p.session_key,
    p.user_pseudo_id,
    p.event_date,
    p.order_ts,
    coalesce(d.channel_group, 'Other')                      as channel_group,  -- wide
    d.device_category,
    d.country,
    p.gross_revenue,
    coalesce(p.refund_value_in_usd, 0.0)                    as refund_value_in_usd,
    p.gross_revenue - coalesce(p.refund_value_in_usd, 0.0)  as net_revenue,
    coalesce(p.shipping_value_in_usd, 0.0)                  as shipping_value_in_usd,
    coalesce(p.tax_value_in_usd, 0.0)                       as tax_value_in_usd,
    p.total_item_quantity,
    p.unique_items
from purch p
left join dims d using (session_key)   -- left join: keep orders even if session dim is missing
```

5.11 dim_channels.sql

```
-- models/marts/core/dim_channels.sql
-- One row per channel_group (exactly 10). is_paid / is_organic / channel_group_order.
-- channel_group strings come from the SINGLE SOURCE OF TRUTH macro (channel_group_case()),
-- enumerated here once as the conformed dimension.
{{ config(materialized='table') }}

with channels as (
    select * from unnest([
        struct('Direct'         as channel_group, false as is_paid, false as is_organic,  1 as channel_group_order),
        struct('Organic Search' as channel_group, false as is_paid, true  as is_organic,  2 as channel_group_order),
        struct('Paid Search'    as channel_group, true  as is_paid, false as is_organic,  3 as channel_group_order),
        struct('Display'        as channel_group, true  as is_paid, false as is_organic,  4 as channel_group_order),
        struct('Paid Social'    as channel_group, true  as is_paid, false as is_organic,  5 as channel_group_order),
        struct('Organic Social' as channel_group, false as is_paid, true  as is_organic,  6 as channel_group_order),
        struct('Email'          as channel_group, false as is_paid, true  as is_organic,  7 as channel_group_order),
        struct('Affiliates'     as channel_group, true  as is_paid, false as is_organic,  8 as channel_group_order),
        struct('Referral'       as channel_group, false as is_paid, true  as is_organic,  9 as channel_group_order),
        struct('Other'          as channel_group, false as is_paid, false as is_organic, 10 as channel_group_order)
    ])
)

select
    to_hex(md5(channel_group))   as channel_key,   -- surrogate PK
    channel_group,
    is_paid,
    is_organic,
    channel_group_order
from channels
```

5.12 fct_cohorts.sql

```
-- models/marts/growth/fct_cohorts.sql
-- Weekly acquisition cohort x age_days for retention (day_1/7/30).
-- cohort_week = week of first touch; age_days = days since cohort start a user was active.
-- Denominator = fixed original cohort size → retention monotonically non-increasing.
{{ config(materialized='table') }}

with first_touch as (
    -- one row per user: the week they were acquired
    select
        user_pseudo_id,
        date_trunc(min(event_date), week(monday))   as cohort_week
    from {{ ref('fct_sessions') }}
    group by user_pseudo_id
),

activity as (
    -- every (user, day) they had a session, with days-since-acquisition
    select
        s.user_pseudo_id,
        ft.cohort_week,
        date_diff(s.event_date, ft.cohort_week, day) as age_days
    from {{ ref('fct_sessions') }} s
```

```
    join first_touch ft using (user_pseudo_id)
    where s.event_date >= ft.cohort_week
),

cohort_size as (
    select cohort_week, count(distinct user_pseudo_id) as cohort_size
    from first_touch
    group by cohort_week
),

active_by_age as (
    select
        cohort_week,
        age_days,
        count(distinct user_pseudo_id) as active_users
    from activity
    group by cohort_week, age_days
)

select
    {{ dbt_utils.generate_surrogate_key(['a.cohort_week','a.age_days']) }} as cohort_key,
    a.cohort_week,
    a.age_days,
    cs.cohort_size,
    a.active_users,
    safe_divide(a.active_users, cs.cohort_size)  as retention_rate
from active_by_age a
join cohort_size cs using (cohort_week)
```

5.13 schema.yml â€" core / finance / growth

The mart tests enforce the three guarantees the agents rely on: **uniqueness** of every grain PK, **referential integrity** of every conformed FK (`relationships`), and the **closed set** of channel groups (`accepted_values` = exactly 10). `persist_docs` pushes descriptions to BigQuery so the catalog is self-documenting.

```
# models/marts/core/core__schema.yml
version: 2

models:
  - name: fct_funnel
    description: "One row per session (PK session_key). Monotonic reached_* flags + session_revenue + wide session dims. Primary session grain for the
      semantic layer."
    config:
      contract: {enforced: true}     # column-level contract on the primary grain
    columns:
      - name: session_key
        tests: [unique, not_null]
      - name: event_date
        tests: [not_null]
      - name: channel_group
        tests:
          - not_null
          - relationships: {to: ref('dim_channels'), field: channel_group}
      - name: user_pseudo_id
        tests:
          - relationships: {to: ref('dim_users'), field: user_pseudo_id}
      - name: reached_purchase
        tests: [not_null]
      - name: session_revenue
        tests:
          - not_null
          - dbt_utils.accepted_range: {min_value: 0}
    # singular tests assert_funnel_monotonicity + assert_session_conversion_rate_bounds
    # live in tests/ and guard the reached_* invariant and 0≤cvr≤1.

  - name: fct_sessions
    description: "One row per session (PK session_key). Engagement + wide session dims + funnel reach."
    columns:
      - name: session_key
        tests: [unique, not_null]
      - name: channel_group
        tests:
          - relationships: {to: ref('dim_channels'), field: channel_group}
      - name: engaged_session
        tests: [not_null]

  - name: fct_daily_funnel
```

```yaml
    description: "Additive daily funnel counts + revenue by event_date x channel/device/country/is_new_user. Rates NOT stored. Aggregates fct_funnel."
    columns:
      - name: daily_funnel_key
        tests: [unique, not_null]
      - name: channel_group
        tests:
          - relationships: {to: ref('dim_channels'), field: channel_group}
      - name: sessions
        tests:
          - not_null
          - dbt_utils.accepted_range: {min_value: 0}
    tests:
      # purchasing_sessions can never exceed sessions (monotonicity at the rollup grain)
      - dbt_utils.expression_is_true:
          expression: "purchasing_sessions ≤ sessions"

  - name: dim_channels
    description: "Conformed channel dimension. Exactly 10 GA4 default channel groups."
    columns:
      - name: channel_key
        tests: [unique, not_null]
      - name: channel_group
        tests:
          - not_null
          - unique
          - accepted_values:
              values: ['Direct','Organic Search','Paid Search','Display',
                       'Paid Social','Organic Social','Email','Affiliates',
                       'Referral','Other']

  - name: dim_users
    description: "One row per user_pseudo_id (PK user_key). Cookie-grain first-touch attrs; user_id null."
    columns:
      - name: user_key
        tests: [unique, not_null]
      - name: user_pseudo_id
        tests: [unique, not_null]

  - name: dim_date
    description: "Conformed date spine 2020-11-01..2021-01-31."
    columns:
      - name: date_key
        tests: [unique, not_null]

  - name: dim_items
    description: "One row per item_id (PK item_key). item_name/brand/category(2..5)/price."
    columns:
      - name: item_key
        tests: [unique, not_null]
```

```yaml
# models/marts/finance/finance__schema.yml
version: 2

models:
  - name: fct_orders
    description: "One deduped row per transaction_id (PK order_key). gross/net/refund/shipping/tax + wide channel/device/date."
    columns:
      - name: order_key
        tests: [unique, not_null]
      - name: session_key
        tests:
          - relationships: {to: ref('fct_sessions'), field: session_key}
      - name: channel_group
        tests:
          - relationships: {to: ref('dim_channels'), field: channel_group}
      - name: gross_revenue
        tests:
          - not_null
          - dbt_utils.accepted_range: {min_value: 0}
      - name: net_revenue
        tests:
          - dbt_utils.accepted_range: {min_value: 0}
    tests:
      - revenue_reconciles:           # custom generic test: mart revenue == raw to the cent
          column_name: gross_revenue
          tolerance: 0

  - name: fct_order_items
    description: "One row per (transaction_id, item line) (PK order_item_key). item_revenue_in_usd, quantity, item dims."
```

```yaml
    columns:
      - name: order_item_key
        tests: [unique, not_null]
      - name: order_key
        tests:
          - relationships: {to: ref('fct_orders'), field: order_key}
      - name: item_key
        tests:
          - relationships: {to: ref('dim_items'), field: item_key}
      - name: item_revenue_in_usd
        tests:
          - dbt_utils.accepted_range: {min_value: 0}
```

```yaml
# models/marts/growth/growth__schema.yml
version: 2

models:
  - name: fct_funnel_by_dim
    description: "Funnel rollup by canonical dimension (long format) for the Decompose mix-vs-rate input."
    columns:
      - name: funnel_by_dim_key
        tests: [unique, not_null]
      - name: dimension_name
        tests:
          - accepted_values:
              values: ['channel_group','device_category','country','is_new_user','landing_page']

  - name: fct_cohorts
    description: "Weekly acquisition cohort x age_days for day_1/7/30 retention."
    columns:
      - name: cohort_key
        tests: [unique, not_null]
      - name: cohort_week
        tests: [not_null]
      - name: retention_rate
        tests:
          - dbt_utils.accepted_range: {min_value: 0, max_value: 1}   # fixed-denominator retention in [0,1]
```

Together these marts are the foundation of the product: wide, conformed, grain-explicit facts that the semantic layer slices without joins, reconciled to raw to the cent, and tested for uniqueness, referential integrity, and the closed 10-channel vocabulary â€” so the agents compose governed metrics over them and never reach raw.

6. Testing Strategy

Helios is an autonomous diagnosis engine: the agents never write SQL, they compose governed metrics that resolve to marts. If a mart is silently wrong, the Critic cannot catch it (the SQL is "valid"), the reconcile guardrail can pass (both sides share the bug), and a confident, well-cited, *wrong* Decision Brief ships. Tests are therefore not hygiene - they are the only thing standing between a transform bug and a fabricated root cause. We run a six-layer pyramid, widest (cheapest, most numerous) at the base:

```
                  â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”
  integration     â”‚  eval harness (50-scenario benchmark)â”‚  1 CI gate
                  â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
  reconciliationâ”‚  test_revenue_reconciles + mcp drift â”‚  ~12 checks
                  â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
  unit            â”‚  sessionize / reached_* / decompose  â”‚  ~15 cases
                  â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
  singular        â”‚  conv-rate bounds, monotonicity      â”‚  ~8 SQL tests
                  â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
  package         â”‚  dbt_utils + dbt_expectations        â”‚  ~60 tests
                  â”œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¤
  generic         â”‚  unique / not_null / accepted_values â”‚  ~150 tests
                  â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜
```

Everything runs under `dbt build`, never `dbt run` then `dbt test` separately - `build` interleaves model and test execution in DAG order so a failed test on `int_ga4__sessionized` aborts before the poisoned data ever reaches `fct_funnel`. Running them separately would materialize the whole graph on bad upstream data and only discover it afterward.

6.1 Generic built-in tests

The four built-ins anchor every model's `schema.yml`. PK uniqueness and not-null on join/grain keys are non-negotiable; `accepted_values` pins the closed vocabularies that the semantic layer depends on; `relationships` enforces the layered DAG's referential integrity.

```yaml
# models/marts/core/_core__models.yml
models:
  - name: fct_funnel
    description: "{{ doc('fct_funnel') }}"
    config:
      contract: {enforced: true}     # column types frozen for the semantic layer
    columns:
      - name: session_key
        data_type: string
        tests:
          - unique
          - not_null
      - name: channel_group
        data_type: string
        tests:
          - not_null
          - accepted_values:
              values: ['Direct','Organic Search','Paid Search','Display',
                       'Paid Social','Organic Social','Email','Affiliates',
                       'Referral','Other']
              config: {severity: error}
          - relationships:
              to: ref('dim_channels')
              field: channel_group
      - name: device_category
        tests:
          - accepted_values: {values: ['desktop','mobile','tablet','smart tv']}
      - name: session_revenue
        tests:
          - not_null:
              config: {where: "reached_purchase"}   # only purchasers must carry revenue
```

`channel_group` is tested with `accepted_values` at the *mart*, not just at `channel_group_case()`, because a macro change plus a stale incremental partition can drift the two apart - we want the failure at the table the agents read.

6.2 Package tests: dbt_utils + dbt_expectations

`packages.yml` pins `dbt-utils`, `dbt-expectations`, and `dbt-project-evaluator` (the latter as a CI lint that fails on un-tested/un-documented models, models reaching across layers, or `fct_` / `dim_` without a PK test). The expectations packages give us range, expression, mutual-exclusivity, and recency assertions that the built-ins lack:

```yaml
  - name: fct_daily_funnel
    columns:
      - name: sessions
        tests:
          - dbt_utils.accepted_range: {min_value: 0, inclusive: true}
      - name: conversion_rate          # not stored, but tested as an expression
    tests:
      # funnel monotonicity at the additive grain (counts, not flags)
      - dbt_utils.expression_is_true:
          expression: "purchasing_sessions ≤ begin_checkout_sessions"
      - dbt_utils.expression_is_true:
          expression: "begin_checkout_sessions ≤ add_to_cart_sessions"
      - dbt_utils.expression_is_true:
          expression: "add_to_cart_sessions ≤ view_item_sessions"
      - dbt_utils.expression_is_true:
          expression: "view_item_sessions ≤ sessions"
      - dbt_expectations.expect_column_pair_values_A_to_be_greater_than_B:
          column_A: revenue
          column_B: 0
          or_equal: true
      # production-only: each day must have a fresh row (no-op on static sample, see Â§7)
      - dbt_utils.recency:
          datepart: day
          field: event_date
          interval: 2
          config:
            severity: "{{ 'error' if target.name == 'prod' else 'warn' }}"

  - name: fct_funnel
    tests:
      # a session cannot be both new and returning user in the same row
```

```yaml
      - dbt_utils.mutually_exclusive_ranges:
          lower_bound_column: 0
          upper_bound_column: 0   # placeholder; see is_new_user expression test below
      - dbt_utils.expression_is_true:
          expression: "not (is_new_user and ga_session_number > 1)"
```

6.3 Singular tests (full SQL)

Singular tests are bespoke SELECTs in `tests/`; any returned row is a failure. These two encode Helios invariants too cross-cutting for a column test.

```sql
-- tests/assert_funnel_monotonicity.sql
-- The reached_* flags are MAX-DOWNSTREAM by construction; any session that
-- violates the chain means the monotonic roll-up logic broke.  Returns the
-- offending sessions (0 rows = pass).
select
    session_key,
    reached_view_item, reached_add_to_cart, reached_begin_checkout,
    reached_add_shipping_info, reached_add_payment_info, reached_purchase
from {{ ref('fct_funnel') }}
where reached_purchase           > reached_add_payment_info
   or reached_add_payment_info   > reached_add_shipping_info
   or reached_add_shipping_info  > reached_begin_checkout
   or reached_begin_checkout     > reached_add_to_cart
   or reached_add_to_cart        > reached_view_item
```

```sql
-- tests/assert_session_conversion_rate_bounds.sql
-- Session→purchase conversion on this GA4 sample sits ~1-4%.  A rate of 0,
-- a rate >100% (impossible), or a wild swing means sessionization or the
-- funnel flags broke.  Bounds are intentionally generous to avoid false
-- alarms on Black Friday / holiday peaks within the window.
with daily as (
    select
        event_date,
        sum(purchasing_sessions) as purchasers,
        sum(sessions)            as sessions
    from {{ ref('fct_daily_funnel') }}
    group by 1
)
select
    event_date,
    purchasers,
    sessions,
    safe_divide(purchasers, sessions) as conv_rate
from daily
where sessions > 0
  and (
        safe_divide(purchasers, sessions) > 1.0       -- impossible
     or safe_divide(purchasers, sessions) < 0.0005    -- effectively zero → broken
     or safe_divide(purchasers, sessions) > 0.20      -- implausibly high → broken
  )
```

`assert_session_conversion_rate_bounds` is also a soft data-quality canary: it is the dbt analogue of eval scenario `06_no_anomaly_control` (a flat funnel that must NOT trip a finding).

6.4 Custom generic test: test_revenue_reconciles

Money is the load-bearing number in every Decision Brief, so we ship a reusable generic test that reconciles any revenue column on any model back to the raw GA4 `purchase_revenue_in_usd` within a tolerance. It lives in `tests/generic/` and is referenced like a built-in.

```sql
-- tests/generic/test_revenue_reconciles.sql
{% test test_revenue_reconciles(model, column_name,
                                source_relation=none,
                                source_column='purchase_revenue_in_usd',
                                tolerance=0.005) %}

{# Sum the mart's revenue and the canonical raw revenue, compare relative drift. #}
{%- set src = source_relation or source('src_ga4', 'events') -%}

with mart_total as (
    select coalesce(sum({{ column_name }}), 0) as amt
    from {{ model }}
),
raw_total as (
```

```
    select coalesce(sum({{ source_column }}), 0) as amt
    from {{ src }}
    where event_name = 'purchase'
),
recon as (
    select
        mart_total.amt as mart_amt,
        raw_total.amt  as raw_amt,
        abs(mart_total.amt - raw_total.amt)
            / nullif(raw_total.amt, 0) as rel_drift
    from mart_total cross join raw_total
)
select *
from recon
where rel_drift > {{ tolerance }}      -- any row returned ⇒ drift exceeds 0.5%
   or (raw_amt = 0 and mart_amt <> 0)  -- mart invented revenue

{% endtest %}
```

```
  - name: fct_orders
    columns:
      - name: gross_revenue
        tests:
          - test_revenue_reconciles:
              tolerance: 0.005
              config: {severity: error}
  - name: fct_funnel
    columns:
      - name: session_revenue
        tests:
          - test_revenue_reconciles
```

This is the in-warehouse twin of `warehouse-mcp.reconcile` (Â§6.6): same 0.5% tolerance, but run at build time so a reconciliation failure blocks the deploy rather than withholding a finding at runtime.

6.5 Unit tests (dbt 1.8+) for the keystone transforms

The KEYSTONE models - `int_ga4__sessionized` and `int_ga4__funnel_steps` - encode logic that **fails silently**: a wrong `MD5` concat order still produces a valid-looking key, a non-monotonic flag still produces a number. Data tests catch these only if a violating row happens to exist in today's data. Unit tests catch them on fixed input, every build, before any real data flows. **Write these first; they are golden-value tests.**

```
# models/intermediate/_int__unit_tests.yml
unit_tests:
  - name: test_sessionize_key_construction
    description: session_key = TO_HEX(MD5(user_pseudo_id - ga_session_id)); deterministic.
    model: int_ga4__sessionized
    given:
      - input: ref('stg_ga4__events')
        rows:
          - {user_pseudo_id: 'u1', ga_session_id: 100, event_name: 'session_start',
             event_timestamp: 1, session_engaged: '1', source: 'google', medium: 'cpc',
             has_gclid: true,  device_category: 'mobile', country: 'US'}
          - {user_pseudo_id: 'u1', ga_session_id: 100, event_name: 'page_view',
             event_timestamp: 2, source: 'google', medium: 'cpc', has_gclid: true}
          - {user_pseudo_id: 'u2', ga_session_id: 100, event_name: 'session_start',
             event_timestamp: 3, source: '(direct)', medium: '(none)', has_gclid: false}
    expect:
      rows:
        # u1+100 collapses 2 events → 1 session; channel from has_gclid+cpc = Paid Search
        - {session_key: "{{ '%s' | format('') }}", user_pseudo_id: 'u1',
           channel_group: 'Paid Search', engaged_session: true}
        - {user_pseudo_id: 'u2', channel_group: 'Direct', engaged_session: false}

  - name: test_reached_flags_are_max_downstream
    description: a session that only fired 'purchase' must back-fill all upstream reached_* flags.
    model: int_ga4__funnel_steps
    given:
      - input: ref('stg_ga4__events')
        rows:
          - {session_key: 's1', event_name: 'purchase',        purchase_revenue_in_usd: 50.0}
          - {session_key: 's2', event_name: 'add_to_cart',     purchase_revenue_in_usd: null}
          - {session_key: 's3', event_name: 'view_item',       purchase_revenue_in_usd: null}
    expect:
      rows:
        # s1 purchased → EVERY upstream flag is true (monotonic), revenue carried
```

```
      - {session_key: 's1', reached_view_item: true, reached_add_to_cart: true,
         reached_begin_checkout: true, reached_add_shipping_info: true,
         reached_add_payment_info: true, reached_purchase: true, session_revenue: 50.0}
      # s2 only added to cart → downstream flags false, upstream true
      - {session_key: 's2', reached_view_item: true, reached_add_to_cart: true,
         reached_begin_checkout: false, reached_purchase: false, session_revenue: 0.0}
      - {session_key: 's3', reached_view_item: true, reached_add_to_cart: false,
         reached_purchase: false, session_revenue: 0.0}

- name: test_decomposition_inputs_additive
  description: fct_daily_funnel step counts must satisfy sessions≥view≥cart≥ ... ≥purchase
               so mix/rate decomposition denominators are well-formed.
  model: fct_daily_funnel
  given:
    - input: ref('fct_funnel')
      rows:
        - {session_key: 'a', event_date: '2021-01-01', channel_group: 'Email',
           device_category: 'desktop', country: 'US', is_new_user: true,
           reached_view_item: true, reached_add_to_cart: true, reached_purchase: true,
           session_revenue: 30.0}
        - {session_key: 'b', event_date: '2021-01-01', channel_group: 'Email',
           device_category: 'desktop', country: 'US', is_new_user: true,
           reached_view_item: true, reached_add_to_cart: false, reached_purchase: false,
           session_revenue: 0.0}
  expect:
    rows:
      - {event_date: '2021-01-01', channel_group: 'Email', device_category: 'desktop',
         country: 'US', is_new_user: true, sessions: 2, view_item_sessions: 2,
         add_to_cart_sessions: 1, purchasing_sessions: 1, revenue: 30.0}
```

These three pin the exact behaviors the agents' decomposition (`mix = Î£ Î”wáµ¢Â·ráµ¢(t0)` , `rate = Î£ wáµ¢(t0)Â·Î”ráµ¢`) depends on. If a refactor breaks sessionization or monotonicity, the unit test fails on synthetic input in <1s, long before the eval gate or production.

6.6 Reconciliation tests vs warehouse-mcp.reconcile

Beyond build-time `test_revenue_reconciles` , CI runs a runtime parity check: for each canonical metric/grain, compare the dbt mart total against `warehouse-mcp.reconcile(metric, grain)` (the independent canonical query the agents trust). Drift must be â‰¤ 0.5%.

```
# ci/reconcile.sh  -- runs after `dbt build`, fails the job on drift
python -m helios.eval.reconcile_check \
  --pairs "revenue:fct_orders,transactions:fct_orders,sessions:fct_funnel,
           conversion_rate:fct_funnel,revenue:fct_daily_funnel" \
  --tolerance 0.005 \
  --fail-on-drift
```

The check guards against the failure mode where both the mart and a hand metric carry the *same* bug: `reconcile` is computed from raw `src_ga4` by a path that does NOT share Helios's macros, so a macro bug surfaces as drift.

6.7 Integration test: the 50-scenario eval harness

The top of the pyramid is the offline benchmark in `eval/scenarios/` (categories `01_single_segment_rate` â€¦ `07_data_quality` , ~50 labeled scenarios). It is the **integration test**: it runs the full Monitorâ†'Decomposeâ†'Diagnoseâ†'Criticâ†'Prescribe chain against fixtures with a known injected root cause and grades the emitted diagnosis. CI promotes it to a **required check** with two hard gates:

- **Root-cause accuracy â‰¥ 85%** (the Diagnose agent names the correct injected cause; naive baseline â‰¤ 45%).
- **Hallucination rate = 0** (no column/metric in any emitted SQL is absent from the semantic registry / GA4 schema).

```
# .github/workflows/ci.yml (excerpt)
  eval-gate:
    needs: [dbt-build]
    steps:
      - run: python -m helios.eval.run --suite all --report eval_report.json
      - run: |
          python -m helios.eval.gate eval_report.json \
            --min-root-cause 0.85 \
            --max-hallucination 0.0   # hard zero
```

Crucially, the **data-quality scenarios (`07_data_quality`) double as data-quality assertions**: each injects a defect (null `transaction_id` , duplicated session, revenue that fails to reconcile, a channel outside the 10 groups) and asserts that Helios *detects and withholds* rather than reports. They are the runtime mirror of Â§6.1â€"6.4's build-time tests - if a generic/singular test is ever weakened, the matching `07_data_quality` scenario starts failing, so the two layers backstop each other.

6.8 Severity, store_failures, selection, and CI mechanics

Mechanism	Helios policy
`severity: error`	All PK uniqueness/not-null, `test_revenue_reconciles`, channel `accepted_values`, both singular tests, all unit tests. Blocks the build.
`severity: warn`	Recency on the static sample (Â§7), wide range checks, freshness on non-prod targets.
`store_failures: true`	Set on the two singular tests + `test_revenue_reconciles` so failing rows land in `helios_eval.dbt_test__audit_*` for triage instead of vanishing.
Tags	`tags: ['keystone']` on sessionize/funnel/reconcile tests; `tags: ['freshness']`, `tags: ['contract']`. Selectable via `dbt build -s tag:keystone`.
`--store-failures-map`	Failures persisted only in CI/prod, not dev, to keep dev cheap.

Severity is environment-aware via `{{ 'error' if target.name == 'prod' else 'warn' }}` so freshness/recency never block a dev iteration on the static sample but hard-fail prod.

CI runs in two tiers:

1. **Slim CI on PRs** - `dbt build -s state:modified+ --defer --state ./prod-manifest`, deferring unbuilt upstreams to the prod manifest so only changed models + descendants build and test. This is the fast path.
2. **Full nightly** - `dbt build` (everything) + `ci/reconcile.sh` + the eval gate.

```
# PR check: build only what changed, defer the rest to prod
dbt build --select state:modified+ \
          --defer --state ./prod-manifest \
          --fail-fast
# nightly + the required eval gate
dbt build && ./ci/reconcile.sh && python -m helios.eval.gate ...
```

Order of authoring (and of trust): **keystone golden-value tests first** (unit + singular), because those bugs are invisible to humans and to the Critic; then reconciliation; then the eval gate as the final, holistic backstop.

---

7. Freshness Strategy

Freshness answers one question for the agents: *is the data recent enough to diagnose against?* A Decision Brief built on a stale or half-loaded partition is worse than no brief - it looks authoritative and is silently wrong. So freshness gates the build, and a stale source aborts rather than producing degraded output.

The honesty up front. Our source is `bigquery-public-data.ga4_obfuscated_sample_ecommerce`, a **static, historical** export covering 2020-11-01 â€¦ 2021-01-31. It does not update. *Real* source freshness on it is meaningless - the newest shard is permanently years old. We therefore design the **production** strategy for a live daily GA4 export and make every freshness check **degrade gracefully to a no-op / informational signal** on the static sample, gated on `target.name`.

7.1 Source freshness (production)

GA4's BigQuery export lands `events_YYYYMMDD` daily, completed each morning UTC (intraday `events_intraday_*` streaming is a Phase-3 item, explicitly out of scope here). We declare freshness on the source with `loaded_at_field` derived from the event timestamp.

```
# models/staging/_src_ga4__sources.yml
sources:
  - name: src_ga4
    database: "{{ var('ga4_project', 'bigquery-public-data') }}"
    schema: ga4_obfuscated_sample_ecommerce
    # PRODUCTION freshness; made informational on the static sample (see config below)
    freshness:
      warn_after:  {count: 36, period: hour}
      error_after: {count: 48, period: hour}
    loaded_at_field: >
      TIMESTAMP_MICROS(MAX(event_timestamp))
    tables:
      - name: events
```

```
        identifier: "events_*"
        # prune date-shards; never scan the full history for a freshness probe
        loaded_at_field: "TIMESTAMP_MICROS(event_timestamp)"
        freshness:
          warn_after:  {count: 36, period: hour}
          error_after: {count: 48, period: hour}
          filter: "_TABLE_SUFFIX ≥ FORMAT_DATE('%Y%m%d', DATE_SUB(CURRENT_DATE(), INTERVAL 5 DAY))"
```

The 36h warn / 48h error thresholds tolerate one missed daily load (e.g. a weekend export hiccup) as a *warning* but treat two consecutive misses as an *error*. The `filter` on `_TABLE_SUFFIX` is essential: a freshness probe must never trigger a full-table scan across the entire shard history - it prunes to the last 5 days so the check costs cents.

Freshness gates the build. CI runs the source check *before* `dbt build`; a hard failure aborts the run.

```
# ci/freshness_gate.sh  (production target only)
if [ "$DBT_TARGET" = "prod" ]; then
  dbt source freshness --select source:src_ga4 || {
      echo "STALE SOURCE: GA4 export missing/late >48h. Aborting build." >&2
      # block: do NOT build marts on stale data; page on-call, do not ship a brief
      ./ci/alert.sh --severity page --reason "ga4-source-stale"
      exit 1
  }
fi
```

On a stale source we block, we do not degrade. The pipeline aborts before any mart materializes, the previous (good) marts remain in place, and on-call is paged. The Orchestrator, seeing no fresh build, runs no diagnosis that day rather than diagnosing on a partial load - failing closed, consistent with the run's reconcile/Critic guardrails.

On the static sample, `dbt source freshness` will always report the shards as years stale, which is correct but useless. We make it informational by scoping the gate to `target.name == 'prod'` (above) and by leaving freshness *declared* but downgraded elsewhere - the check still documents the production SLA in `dbt docs` and runs as a no-op signal in dev.

7.2 Model-level freshness / build_after (dbt 1.9)

dbt 1.9 lets models declare their own freshness/`build_after` so the scheduler rebuilds a model only when its inputs are fresh, instead of on a blind cron. We apply it to the keystone facts:

```
  - name: fct_daily_funnel
    config:
      freshness:
        build_after: {count: 6, period: hour}   # rebuild only if >6h since last build AND source is fresh
        updates_on: any   # rebuild when ANY upstream (events shard) advances
```

This couples the Monitor/Decompose feed (`fct_daily_funnel`) to source arrival: it rebuilds shortly after the morning export lands, not on a fixed wall-clock that might fire before the data exists. On the static sample `build_after` simply never re-triggers (no new shards), which is the correct no-op.

7.3 Per-layer freshness SLAs

Freshness propagates down the DAG with widening tolerance - staging must be as fresh as the source; marts inherit source freshness plus their build cadence; the semantic layer is only as fresh as the marts it reads.

Layer	Model(s)	Refresh cadence (prod)	Warn after	Error after	On breach	Static-sample behavior
source	`src_ga4.events_*`	daily, ~morning UTC	36h	48h	block build + page	informational no-op
staging	`stg_ga4__*` (view)	on read	inherits source	inherits source	n/a (views)	inherits source
intermediate	`int_ga4__sessionized`, `int_ga4__funnel_steps`	per build (ephemeral/view)	36h	48h	abort downstream	no-op
marts/core	`fct_funnel`, `fct_sessions`, `fct_daily_funnel`	daily incremental, ~07:00 UTC	30h	48h	hold marts; page	recency = warn

Layer	Model(s)	Refresh cadence (prod)	Warn after	Error after	On breach	Static-sample behavior
marts/finance,growth	`fct_orders`, `fct_order_items`, `fct_funnel_by_dim`, `fct_cohorts`	daily table	30h	48h	hold marts	recency = warn
dims	`dim_users/items/channels/date`	daily / on change (`snap_dim_items` SCD2)	7d	14d	warn	warn
semantic	`semantic_layer.yaml` â†' semantic-mcp	reads marts	= marts	= marts	refuse query if marts stale	n/a

`dim_date` is a fixed conformed spine over the dataset window, so it has no meaningful freshness SLA (it is regenerated only when the window changes).

7.4 Partition-lateness handling

GA4 occasionally back-fills a shard: yesterday's `events_YYYYMMDD` can receive late hits hours after first landing. A naive "only build today's partition" incremental would permanently miss those rows. Our incremental config absorbs this with a **3-day lookback** that re-materializes recent partitions every run:

```
{{ config(
    materialized='incremental',
    incremental_strategy='insert_overwrite',
    partition_by={'field': 'event_date', 'data_type': 'date', 'granularity': 'day'},
    cluster_by=['device_category', 'channel_group'],
    require_partition_filter=true
) }}

select ...
from {{ ref('stg_ga4__events') }}
{% if is_incremental() %}
  -- reprocess the trailing 3 days so late-arriving shards correct prior partitions
  where event_date ≥ date_sub(current_date(), interval 3 day)
{% endif %}
```

`insert_overwrite` atomically replaces those 3 day-partitions, so late hits land and any earlier double-count is overwritten, not appended. Three days covers GA4's typical back-fill window with margin; a shard that arrives >3 days late is rare and is caught instead by the recency test (Â§7.5) flagging a gap. On the static sample `is_incremental()` is false on first build and the lookback predicate is simply never exercised.

7.5 Recency tests

Freshness (a `dbt source freshness` concept) checks *source* lag; recency tests assert that the *built marts* actually contain a recent row - catching the case where the source was fresh but the build silently skipped a partition.

```
- name: fct_daily_funnel
  tests:
    - dbt_utils.recency:
        datepart: day
        field: event_date
        interval: 2
        config:
          severity: "{{ 'error' if target.name == 'prod' else 'warn' }}"
    # row-count recency: today's partition must carry a plausible volume
    - dbt_expectations.expect_table_row_count_to_be_between:
        min_value: 1
        row_condition: "event_date = date_sub(current_date(), interval 1 day)"
        config:
          severity: "{{ 'error' if target.name == 'prod' else 'warn' }}"
```

`dbt_utils.recency` errors in prod if the newest `event_date` is more than 2 days old; the `dbt_expectations` row-count-recency test errors if yesterday's partition exists but is empty (a half-loaded shard). Both are scoped to `severity: warn` off prod, so on the **static sample they degrade to warnings** - the analyst sees an informational "newest partition is 2021-01-31, expected within 2 days" notice without the build failing. This is the deliberate graceful-degradation contract: the production SLAs are fully declared and visible in `dbt docs`, run as hard gates in prod, and become benign informational signals on the historical public dataset.

8. Lineage Strategy

Lineage is not a diagram Helios draws after the fact; it is a property the dbt project *enforces by construction*. Every edge in the DAG exists because one model named another with `ref()` or named a source with `source()`. Because the agents never hand-write SQL â€" they compose governed metrics that resolve to marts that resolve, transitively, back to raw GA4 columns â€" that unbroken `ref()` / `source()` chain is literally the artifact that proves the product's headline guarantee: **0 hallucinated columns, 100% governed SQL**. This section specifies how the DAG is built, declared, governed, and traced end-to-end into the semantic layer.

8.1 `ref()` and `source()` build the DAG â€" never hard-code table names

A relation is referenced exactly two ways, and physical dataset/table names appear in exactly one place each.

```
-- staging/stg_ga4__events.sql : the ONLY place the raw shards are named (via source())
with raw as (
    select * from {{ source('src_ga4', 'events') }}   -- resolves to bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*
    where {{ ga4_shard_filter() }}                      -- prunes _TABLE_SUFFIX shards
)
...

-- intermediate/int_ga4__sessionized.sql : references the staging model, never the raw table
with events as (
    select * from {{ ref('stg_ga4__events') }}
),
params as (
    select * from {{ ref('stg_ga4__event_params') }}
)
...

-- marts/core/fct_funnel.sql : references intermediate, never staging or source
select * from {{ ref('int_ga4__funnel_steps') }}
  join {{ ref('int_ga4__sessionized') }} using (session_key)
```

Rules that keep lineage truthful:

- **`source()` appears only in staging.** `src_ga4.events` is the single declared upstream. No model below staging may `source()`; `dbt_project_evaluator`'s `fct_source_fanout` / `fct_staging_dependent_on_source` flags any violation in CI.
- **`ref()` everywhere else.** dbt resolves `ref()` to the environment-correct relation (`helios_dev.*` vs `helios_prod.*`) at compile time, so the same SQL is portable across dev/CI/prod. Hard-coding `helios_prod.fct_funnel` would (a) break the DAG edge â€" dbt wouldn't know to build the parent first â€" and (b) silently run dev models against prod data. Both are CI failures.
- **One direction only.** `source → staging → intermediate → marts → semantic`. No model `ref()`s a model in its own or a downstream layer. The layered contract is enforced by `dbt_project_evaluator` (`fct_model_directory_get`, `fct_rejoining_of_upstream_concepts`) plus naming-convention checks.

The payoff: the DAG is *derived*, not maintained. `dbt parse` reads the `ref()` / `source()` calls and writes the graph into `manifest.json`. Nobody edits a lineage file.

8.2 Exposures â€" the 7 agents + the Decision Brief as declared downstream consumers

dbt's DAG normally ends at marts. Helios extends it one hop further with **exposures**, which declare the non-dbt consumers so that "what feeds the Decision Brief?" and "what breaks if I change `fct_funnel`?" have machine-checkable answers. The seven agents and the Decision Brief are each an exposure whose `depends_on` lists the marts (and the semantic models) they consume through `semantic-mcp`.

```
# models/exposures/exposures.yml
version: 2

exposures:
  - name: helios_orchestrator
    label: "Agent: Orchestrator (Opus)"
    type: application
    maturity: high
    url: https://helios.internal/agents/orchestrator
    description: >
      Plan-execute-critique controller for the autonomous run. Reads no marts directly;
      depends on the semantic layer transitively through every worker agent.
    depends_on:
      - ref('fct_daily_funnel')
      - ref('semantic_layer')          # the MetricFlow semantic models node
    owner: {name: Helios AE Team, email: ae@pristineforests.com}
    meta: {agent_model: claude-opus, mcp_allowlist: [semantic, stats, report]}
```

```yaml
- name: helios_monitor
  label: "Agent: Monitor (Sonnet)"
  type: application
  maturity: high
  url: https://helios.internal/agents/monitor
  description: "Detects funnel anomalies day-over-day; feeds Decompose. Consumes the additive daily grain."
  depends_on: [ref('fct_daily_funnel')]
  owner: {name: Helios AE Team, email: ae@pristineforests.com}

- name: helios_decompose
  label: "Agent: Decompose (Sonnet)"
  type: application
  maturity: high
  url: https://helios.internal/agents/decompose
  description: "Mix vs rate vs interaction decomposition of funnel deltas via stats-mcp."
  depends_on: [ref('fct_daily_funnel'), ref('fct_funnel_by_dim')]
  owner: {name: Helios AE Team, email: ae@pristineforests.com}

- name: helios_diagnose
  label: "Agent: Diagnose (Opus)"
  type: application
  maturity: high
  url: https://helios.internal/agents/diagnose
  description: "Root-cause hypothesis ranking; slices fct_funnel/fct_sessions by canonical dims."
  depends_on: [ref('fct_funnel'), ref('fct_sessions'), ref('fct_cohorts')]
  owner: {name: Helios AE Team, email: ae@pristineforests.com}

- name: helios_prescribe
  label: "Agent: Prescribe (Sonnet)"
  type: application
  maturity: medium
  url: https://helios.internal/agents/prescribe
  description: "Powered experiment backlog from observed rates/variances."
  depends_on: [ref('fct_funnel'), ref('fct_orders')]
  owner: {name: Helios AE Team, email: ae@pristineforests.com}

- name: helios_critic
  label: "Agent: Critic (Opus)"
  type: application
  maturity: high
  url: https://helios.internal/agents/critic
  description: "Adversarial gate; re-checks reconciliation and significance before a finding ships."
  depends_on: [ref('fct_funnel'), ref('fct_orders'), ref('fct_daily_funnel')]
  owner: {name: Helios AE Team, email: ae@pristineforests.com}

- name: helios_narrator
  label: "Agent: Narrator (Sonnet)"
  type: application
  maturity: medium
  url: https://helios.internal/agents/narrator
  description: "Renders governed numbers to prose. No run_query; reads only finalized findings."
  depends_on: [ref('fct_daily_funnel')]
  owner: {name: Helios AE Team, email: ae@pristineforests.com}

- name: helios_decision_brief
  label: "Helios Decision Brief"
  type: analysis
  maturity: high
  url: https://helios.internal/briefs/latest
  description: >
    The executive deliverable: anomaly → decomposition → diagnosis → dollar-at-risk →
    prescribed backlog. The terminal consumer of the entire DAG.
  depends_on:
    - ref('fct_funnel')
    - ref('fct_daily_funnel')
    - ref('fct_funnel_by_dim')
    - ref('fct_orders')
    - ref('fct_cohorts')
  owner: {name: Helios Product, email: product@pristineforests.com}
```

`maturity` (`high` / `medium` / `low`) signals consumer stability in the docs site; `type` (`application` / `analysis` / `dashboard` / `ml`) categorizes them. These nodes appear in `dbt docs` and are first-class selection targets (8.4).

8.3 Column-level lineage, the docs DAG, and the lineage artifacts

Model-level lineage is free from `ref()`. **Column-level lineage** (which raw GA4 column ultimately feeds `revenue`?) is reconstructed from two artifacts dbt emits on every run:

- **`target/manifest.json`** â€" the full graph: every node, its `depends_on.nodes`, its compiled SQL, its tests, contracts, exposures, and `columns` with descriptions. This is the lineage source of truth.
- **`target/catalog.json`** â€" produced by `dbt docs generate`; the *physical* schema (column names, types, row/byte stats) read back from BigQuery's `INFORMATION_SCHEMA`. The doc site overlays catalog onto manifest so a column shows both its declared description and its warehouse-confirmed type.

Column-level lineage (manifest + compiled-SQL parsing) lets a reviewer trace `revenue` (semantic metric) -> `fct_orders.gross_revenue` -> `stg_ga4__events.purchase_revenue_in_usd` -> `src_ga4.events.ecommerce.purchase_revenue_in_usd`. The hosted-dbt and OSS tools (`sqlglot`-based column lineage, `dbt docs` lineage graph) walk exactly this chain. The static docs DAG is generated by:

```
dbt docs generate           # builds manifest.json + catalog.json
dbt docs serve --port 8080  # interactive lineage graph (--select / --exclude, +N/N+ ancestry)
```

In the served graph, clicking `fct_funnel` highlights its full ancestry (`+fct_funnel`) and descendants including the exposures (`fct_funnel+`). The same `+model` / `model+` operators that drive the graph also drive selection (next).

8.4 Impact analysis with state-based selection

Before any change ships, Helios computes its blast radius against the **deferred prod manifest** (the `manifest.json` from the last prod build, the "state").

```
# What did I change, and everything downstream (children), incl. exposures?
dbt ls --select state:modified+ --state ./prod-manifest

# Everything UPstream of a model (its dependencies) â€" what must build first?
dbt ls --select +fct_funnel

# A model + its full descendant cone (what a change to it can break)
dbt ls --select fct_funnel+

# Does this change reach the Decision Brief?
dbt ls --select state:modified+,exposure:helios_decision_brief --state ./prod-manifest

# Slim CI: build only changed nodes + children, deferring unbuilt parents to prod
dbt build --select state:modified+ --defer --state ./prod-manifest --favor-state
```

`state:modified` compares node fingerprints (compiled SQL, configs, contracts, macro hashes) to the stored state; the `+` operator extends the selection to descendants. `--defer` means unselected parents are read from their *prod* relations rather than rebuilt, so a one-line change to `int_ga4__sessionized` rebuilds only the sessionization keystone and its cone â€" not the full three-month history â€" keeping the CI run inside the 5 GiB byte budget. This is the operational definition of "impact analysis": the set returned by `state:modified+` is exactly the set of things that can change behavior, and intersecting it with `exposure:*` tells you which agents/briefs to re-eval.

8.5 Groups, access, and contracts â€" governance controls on the lineage edges

Lineage tells you *what* connects; groups/access/contracts decide *who may connect and what they may rely on*. These are the mart-protection layer.

Groups + access partition the project into ownership domains and restrict cross-domain `ref()`.

```
# models/marts/__marts.yml
groups:
  - name: core
    owner: {name: Helios AE Team, email: ae@pristineforests.com}
  - name: finance
    owner: {name: Helios Finance, email: finance@pristineforests.com}

models:
  - name: int_ga4__sessionized
    group: core
    access: private        # keystone: nothing outside `core` may ref() it
  - name: fct_funnel
    group: core
    access: public         # the semantic layer's primary session grain â€" stable, depended-on
  - name: fct_orders
    group: finance
    access: protected      # ref-able only within the project (not across dbt Mesh projects)
```

`access` has three levels: **private** (referenceable only within the same group â€" used for the `int_ga4__*` keystones so no mart bypasses them), **protected** (same dbt project only, the default), **public** (any project, including downstream Mesh projects). Marking `int_ga4__sessionized` private structurally forbids a careless `ref('int_ga4__sessionized')` from a finance mart that should have gone through `fct_funnel`; dbt raises a parse-time error. This is governance encoded in the DAG.

Contracts freeze a model's output schema so downstream consumers (and the semantic layer) cannot be silently broken by an upstream column rename or retype.

```
  - name: fct_funnel
    config: {contract: {enforced: true}}
    columns:
      - name: session_key
        data_type: string
        constraints: [{type: not_null}, {type: primary_key}]
      - name: reached_purchase
        data_type: boolean
        constraints: [{type: not_null}]
      - name: session_revenue
        data_type: numeric
```

With `contract.enforced: true`, dbt validates the built relation's columns/types against this declaration *before* it replaces the prod table â€" a `BREAKING CHANGE` error if `session_revenue` were dropped or its type changed. Because `semantic_layer.yaml` binds metrics to these exact columns, the contract is the structural guarantee behind "0 hallucinated columns": the semantic layer can only reference columns the contract promises exist.

8.6 Cross-project lineage via dbt Mesh (Phase 3, multi-tenant / warehouse-agnostic)

The public sample is single-project. The production design (Bible Phase 3) is multi-tenant: a shared `helios_core` dbt project owns staging/intermediate/conformed dims, and each tenant (or each warehouse adapter â€" BigQuery, Snowflake, Databricks) is a **downstream project** that consumes `helios_core`'s `public` models via cross-project `ref()`.

```
# tenant project: dependencies.yml
projects:
  - name: helios_core            # the upstream producer project

# tenant model
select * from {{ ref('helios_core', 'fct_funnel') }}   -- cross-project ref, version-pinned
```

dbt Mesh stitches the two projects' manifests so lineage and impact analysis span the boundary: a `state:modified+` on `helios_core.fct_funnel` reports the *tenant* exposures it breaks. Only `public`-access models are ref-able across the boundary â€" which is precisely why `int_ga4__*` is `private` and `fct_funnel` is `public`. **Model versions** (`fct_funnel.v1`, `.v2`) let `helios_core` ship a breaking change while tenants migrate on their own schedule. The honest caveat for the static sample: Mesh is designed but not exercised â€" there is one project and one warehouse today; the access/version annotations are in place so Phase 3 is a configuration change, not a rewrite.

8.7 Lineage extending into the semantic layer â€" the chain that proves "0 hallucinated columns"

The DAG does not stop at marts. `semantic_layer.yaml` defines MetricFlow semantic models whose `model:` points at a dbt `ref()` and whose measures/dimensions name *columns of that ref*. Because every entity, dimension, and measure binds to a `ref()`d mart column, MetricFlow's parse step fails if a metric references a column the mart (and its contract) does not expose.

```
# semantic_layer.yaml (MetricFlow) â€" bound 1:1 to the dbt DAG
semantic_models:
  - name: funnel
    model: ref('fct_funnel')              # ← the lineage edge into the semantic layer
    entities: [{name: session, type: primary, expr: session_key}]
    measures:
      - {name: sessions, agg: count_distinct, expr: session_key}
      - {name: purchasing_sessions, agg: sum, expr: "if(reached_purchase, 1, 0)"}
      - {name: revenue, agg: sum, expr: session_revenue}
metrics:
  - name: session_conversion_rate
    type: ratio
    type_params: {numerator: purchasing_sessions, denominator: sessions}
```

The complete provable chain for one metric:

```
session_conversion_rate           (semantic_layer.yaml metric, composed by semantic-mcp)
  â””â”€ measures purchasing_sessions / sessions
       â””â”€ expr over fct_funnel.reached_purchase, fct_funnel.session_key   (contract-enforced columns)
            â””â”€ ref('int_ga4__funnel_steps') reached_* + ref('int_ga4__sessionized') session_key
                 â””â”€ ref('stg_ga4__events') / ref('stg_ga4__event_params')
                      â””â”€ source('src_ga4','events')  =  raw GA4 events_* columns
```

Every hop is a `ref()` / `source()` / `expr` binding recorded in `manifest.json` (dbt) and the MetricFlow `semantic_manifest.json`. When an agent asks `semantic-mcp.build_query('session_conversion_rate', dims=['channel_group'])`, the only columns that can appear are ones this chain validates back to raw GA4. An LLM cannot inject a column that is not in the manifest â€” the metric simply will not compile. That is what "0 hallucinated columns" means operationally: it is enforced by lineage, not promised by a prompt.

---

9. Documentation Strategy

Documentation in Helios has two distinct registers that must never blur: **technical docs** (what a column physically is â€” owned by dbt) and **business definitions** (what a metric *means* to the business â€” owned by the semantic layer). Both are version-controlled, both are required, and both are enforced in CI. A model that lacks a description, a test, or a contract does not merge.

9.1 schema.yml descriptions on every model and every column

Every model file is paired with a `schema.yml` carrying a model-level description and a description for *every* column. Descriptions reference reusable **doc blocks** (`{{ doc( ... ) }}`) so a definition lives once and is cited everywhere.

```yaml
# models/marts/core/_core__models.yml
version: 2

models:
  - name: fct_funnel
    description: '{{ doc("fct_funnel") }}'
    group: core
    access: public
    config:
      contract: {enforced: true}
      persist_docs: {relation: true, columns: true}
    meta:
      owner: ae@pristineforests.com
      maturity: high
      contains_pii: false
      sla: "refreshed by 06:00 UTC; freshness N/A on static public sample"
      grain: "one row per session_key"
    columns:
      - name: session_key
        description: '{{ doc("session_key") }}'
        data_type: string
        constraints: [{type: not_null}, {type: primary_key}]
        tests: [unique, not_null]
      - name: reached_purchase
        description: '{{ doc("reached_flag") }} Stage: purchase (the terminal funnel step).'
        data_type: boolean
        tests: [not_null]
      - name: channel_group
        description: '{{ doc("channel_group") }}'
        tests:
          - relationships: {to: ref('dim_channels'), field: channel_group}
      - name: session_revenue
        description: "Total purchase_revenue_in_usd attributed to the session (USD; 0 for non-purchasing sessions)."
        data_type: numeric
```

Note the column descriptions are not prose duplicates â€” `session_key`, `reached_flag`, and `channel_group` are *shared* doc blocks, so the monotonic-funnel and session-key conventions are defined once and rendered on every model that surfaces them.

9.2 Doc blocks â€” reusable definitions in `.md` files

Doc blocks live in `.md` files alongside the models and are the single source of *technical* prose. Defining the funnel-monotonicity rule once means it cannot drift between `fct_funnel`, `fct_daily_funnel`, and `fct_funnel_by_dim`.

```
{# models/marts/core/_core__docs.md #}
```

```
{% docs session_key %}
Surrogate session identifier: `TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))`.
The canonical session grain. `sessions = COUNT(DISTINCT session_key)` â€” never `COUNT(*)`,
never `FARM_FINGERPRINT`. Cookie-scoped (no logged-in `user_id` in the GA4 sample).
{% enddocs %}

{% docs reached_flag %}
A **max-downstream (monotonic)** funnel flag: TRUE if the session reached this stage **or any
later stage**. This guarantees `sessions ≥ reached_view_item ≥ ... ≥ reached_purchase`, so
every step-to-step rate is ≤ 1 by construction. The retired `did_*` naming is forbidden.
{% enddocs %}

{% docs channel_group %}
One of exactly **10** GA4 default channel groups (Direct, Organic Search, Paid Search, Display,
Paid Social, Organic Social, Email, Affiliates, Referral, Other). Derived in the single source of
truth `channel_group_case()` macro from session-scoped source/medium (with `has_gclid` paid
detection), falling back to user first-touch `traffic_source` only when session scope is null.
{% enddocs %}

{% docs fct_funnel %}
One row per session (PK `session_key`). Carries the monotonic `reached_*` flags, `session_revenue`,
and wide session dimensions (channel_group, device_category, country, is_new_user). This is the
semantic layer's **primary session grain** â€” `fct_daily_funnel` aggregates *this* model (so it
inherits revenue), not `int_ga4__funnel_steps`.
{% enddocs %}
```

9.3 persist_docs â€“ pushing descriptions into BigQuery metadata

`persist_docs: {relation: true, columns: true}` (set project-wide and shown per-model above) writes each model's description to the BigQuery **table description** and each column's description to the **column description** at build time. The benefit: an analyst inspecting `helios_prod.fct_funnel` in the BigQuery console â€” or any catalog tool reading `INFORMATION_SCHEMA.COLUMN_FIELD_PATHS` â€” sees the same governed definition without opening the repo. Documentation stops being a thing you have to remember to read.

```
# dbt_project.yml â€” project-wide default
models:
  helios:
    +persist_docs:
      relation: true
      columns: true
```

9.4 dbt docs generate / serve and the static site

```
dbt docs generate           # manifest.json + catalog.json (catalog reads INFORMATION_SCHEMA)
dbt docs serve              # local interactive site: search, lineage graph, column types
```

In CI the generated `target/` (the static `index.html`, `manifest.json`, `catalog.json`) is published to GitHub Pages / GCS so the docs site is always live and matches `main`. The site renders: model + column descriptions (from schema.yml / doc blocks), the lineage DAG (Â§8.3), tests per model, contracts, and the exposures with their `maturity` / `owner`. `meta` fields surface as a properties table.

9.5 Model-level meta â€“ owner, maturity, contains_pii, sla

Every model carries structured `meta` (shown in 9.1) so governance is queryable, not tribal:

Field	Purpose	Example
`owner`	Who is paged when it breaks	`ae@pristineforests.com`
`maturity`	Stability signal to consumers	`high` (marts) / `medium` (growth rollups)
`contains_pii`	Drives access policy + masking review	`false` (GA4 sample is obfuscated; `user_pseudo_id` is a cookie hash)
`sla`	Freshness/availability promise	"06:00 UTC daily; **N/A on static sample**"

The PII flag is honest about the sample: the obfuscated GA4 export has no real identifiers, so `contains_pii: false` is correct *today*, but the field exists so the multi-tenant production export (real `user_id`, geo) can flip it and trigger the masking-policy review without a schema redesign.

9.6 Two registers: semantic layer = business definitions, dbt = technical

The division of labor is strict and is the answer to "where is the *real* definition of `revenue_per_session`?":

- **`semantic_layer.yaml` `business_definition` / `label` fields are canonical for what a metric MEANS.** Example: `revenue_per_session` â€" "*Total purchase revenue (USD) divided by total sessions over the window; the headline efficiency metric; computed as SUM(revenue)/SUM(sessions) after grouping, never an average of per-segment ratios.*" This is what the Narrator quotes and what the Critic checks a finding against.
- **dbt schema.yml / doc blocks are canonical for what a column IS** â€" its type, its grain, its derivation, its source column. Technical, not interpretive.

They are linked by lineage (Â§8.7): the metric's `business_definition` sits atop a measure that binds to a contract-enforced mart column that traces to raw GA4. Business meaning and physical truth are documented in different files but provably the same object.

9.7 README / runbook

The repo root `README.md` is the operational front door: setup (`dbt deps` , profile/WIF auth), the build commands (`dbt build` , layer selection, `--full-refresh` for the small sample), the env vars (`GOOGLE_APPLICATION_CREDENTIALS` , `HELIOS_WH_TOKEN`), and a **runbook** section â€" how to respond to a failed freshness check (N/A on the static sample; real on the live export), a contract `BREAKING CHANGE` , or a reconciliation failure (`revenue_reconciles` > 0.5% drift). The runbook links each failure mode to the owning `meta.owner` and the relevant singular test (`assert_funnel_monotonicity` , `assert_session_conversion_rate_bounds`).

9.8 Governance â€" CODEOWNERS, PR review, docs-in-lockstep

- `CODEOWNERS` maps paths to the `group` owners so a change to `models/marts/finance/**` requires Finance review and `semantic_layer.yaml` requires AE review. This mirrors the dbt `groups` (Â§8.5) â€" ownership is the same in Git and in the DAG.
- **PR review** is mandatory; CI must be green (build + tests + the doc/coverage gate below + the eval gate).
- **Docs-in-lockstep rule (CLAUDE.md):** when an artifact is added or a canonical name changes, the same PR updates `DEPENDENCY_MAP.md` , the Bible's Reference Card, and `CLAUDE.md` . A metric rename that doesn't touch `semantic_layer.yaml` 's `business_definition` and the dbt doc block in the same commit is a review reject. Documentation drift is treated as a build break, not a nicety.

9.9 Enforcing doc + test + contract coverage in CI

Coverage is not aspirational; `dbt_project_evaluator` fails the build when a model is undocumented, untested, or uncontracted.

```
# dbt_project.yml â€" make coverage rules hard failures
vars:
  dbt_project_evaluator:
    documentation_coverage_target: 100      # every model + column described
    test_coverage_target: 100               # every model has ≥1 test
    primary_key_test_coverage_target: 100   # every model's PK is unique + not_null
models:
  dbt_project_evaluator:
    +severity: error                        # findings fail CI, not just warn
```

```
# CI step (runs against the slim, state:modified+ selection)
dbt build --select package:dbt_project_evaluator --resource-type test
# evaluator surfaces: fct_undocumented_models, fct_models_without_tests,
#                     fct_missing_primary_key_tests, fct_undocumented_columns,
#                     fct_models_without_contracts (custom rule), fct_source_fanout
```

The contract requirement is added as a project rule: every `fct_*` / `dim_*` mart must set `contract.enforced: true` , so a new mart cannot reach the semantic layer (and therefore the agents) without a frozen, documented, tested schema. Combined with the eval gate (no accuracy regression, zero hallucination) and the lineage chain of Â§8.7, the documentation strategy closes the loop: a column an agent can reference is, by CI construction, a column that is described, typed, contracted, tested, and traceable to raw GA4.

10. Production-Readiness Checklist

A tickable gate before calling the dbt layer production-grade. Group it into the CI pipeline (`dbt build` + tests + `dbt_project_evaluator` + the eval gate) so "production-ready" is *enforced*, not aspirational.

Project & configuration

- ☐ `require-dbt-version: "≥1.7.0"` pinned; `packages.yml` versions pinned and `dbt deps` reproducible.
- ☐ `dbt_project.yml` sets folder-level materializations, `partition_by` `event_date` , `cluster_by` , `insert_overwrite` , `persist_docs` , `+group` / `+access` , `+tags` .

- [ ] `profiles.yml`: `dev â†' helios_dev`, `prod â†' helios_prod`; prod auth via Workload Identity Federation (no JSON keys); `maximum_bytes_billed` set per target.
- [ ] `vars` centralize the build window, incremental lookback (3 days), engagement threshold (10000 ms).

Sources

- [ ] All raw access via `source()` â€" no model refs a raw table directly.
- [ ] `source freshness` configured (warn 36h / error 48h) and gates the build in prod; informational-only on the static sample.
- [ ] Date-sharded wildcard pruned via `_TABLE_SUFFIX`; build window bounded by vars.

Staging

- [ ] One staging model per source entity; views; rename/cast/light-flatten only â€" no joins/aggregations/ `SELECT *`.
- [ ] `get_event_param`, `sessionize`, `channel_group_case` macros are the single source of those transforms.
- [ ] Keys tested `unique` + `not_null`.

Intermediate (keystones)

- [ ] `int_ga4__sessionized` and `int_ga4__funnel_steps` have **golden-value unit tests** (these fail silently).
- [ ] Funnel monotonicity holds (`assert_funnel_monotonicity`); `session_key` unique per session.
- [ ] `traffic_source` first-touch fallback documented and tested.

Marts

- [ ] Every mart has a single declared grain + primary-key uniqueness test.
- [ ] Conformed dims (`dim_date`, `dim_channels`, `dim_users`, `dim_items`) shared across facts; `relationships` tests pass.
- [ ] `dim_channels` `accepted_values` = exactly the 10 channel groups.
- [ ] Marts are wide; the 5 semantic grains (`fct_funnel`, `fct_sessions`, `fct_orders`, `fct_order_items`, `fct_cohorts`) exist and reconcile.
- [ ] Incremental facts re-materialize the 3-day lookback correctly on re-run (idempotent).

Testing

- [ ] `dbt build` (not separate `run` + `test`); generic + singular + custom-generic + unit tests present.
- [ ] `revenue_reconciles` to the cent; `session_conversion_rate` bounded [0,1].
- [ ] Reconciliation to `warehouse-mcp.reconcile` within 0.5%.
- [ ] The 50-scenario eval benchmark runs as the CI integration test; gate: root-cause â‰¥85%, hallucination = 0.
- [ ] `store_failures` enabled; severities (warn vs error) deliberate.

CI/CD

- [ ] Slim CI on PRs: `state:modified+` with `--defer` to the prod manifest; full build on merge to `main`.
- [ ] `dbt_project_evaluator` enforces: every model documented, tested, has a contract, correct layer refs.
- [ ] Prod build scheduled after source freshness passes.

Freshness & cost

- [ ] Per-layer freshness SLAs defined; stale source blocks the run + alerts (prod).
- [ ] `maximum_bytes_billed` caps every job; partition pruning verified; no `SELECT *` in built models.
- [ ] Per-run byte budget (â‰¤5 GiB) honored end-to-end.

Lineage & governance

- [ ] `exposures.yml` declares the 7 agents + the Decision Brief; `manifest.json` / `catalog.json` published.
- [ ] Model `groups` + `access` (staging/intermediate private, marts public) enforced; mart `contracts` enforce schemas for downstream consumers.
- [ ] Lineage traces a metric â†' mart â†' raw GA4 column (the "0 hallucinated columns" chain).

Documentation

- [ ] Every model + column has a description; doc blocks for reusable definitions; `persist_docs` pushes to BigQuery metadata.
- [ ] `dbt docs` site generated in CI; model `meta` carries owner/maturity/contains_pii.
- [ ] Docs kept in lockstep with the Bible / CLAUDE.md / `semantic_layer.yaml`.

Production-frontier (Bible Phase 3+, deferred)

- [ ] Multi-tenant isolation (per-tenant datasets + byte budgets); warehouse-agnostic adapters behind the semantic layer.
- [ ] Intraday/streaming ingestion (replaces the static daily sample); cross-project lineage via dbt Mesh.